შესავალი QGIS-ის პროგრამირებაში

C++ PyQGIS-იდან QGIS Core-მდე

გიორგი კაპანაძე

თბილისი · 2026

სარჩევი

01 **შესავალი: რა არის QGIS-ის პროგრამირება** 5

01.1 რა არის პროგრამირება და რას ვსწავლობთ
01.2 PyQGIS და C++ API: ერთი ბირთვი, ორი ინტერფეისი
01.3 QGIS-ის არქიტექტურა
01.4 ერთი და იგივე ამოცანა ორ ენაზე
01.5 როგორ მუშაობს კომპილაცია
01.6 სამუშაო გარემოს მომზადება
01.7 პირველი C++ პროგრამა
01.8 Python vs C++ ერთი შეხედვით
01.9 სავარჯიშო და შეჯამება

02 **ტიპები და ცვლადები** 20

02.1 რა არის ცვლადი
02.2 ცვლადის შექმნა
02.3 ძირითადი ტიპები
02.4 კოდი შედეგით
02.5 const და auto
02.6 მთელი რიცხვების გაყოფა
02.7 სიზუსტე და გარდაქმნა
02.8 სავარჯიშო და შეჯამება

03 **ფუნქციები და namespace** 28

03.1 რა არის ფუნქცია
03.2 ფუნქციის ანატომია
03.3 namespace — სახელების საქაღალდე
03.4 overload — ერთი სახელი, სხვადასხვა პარამეტრები
03.5 default პარამეტრი
03.6 მანძილი ორ წერტილს შორის
03.7 რამდენიმე ფუნქცია ერთ ფაილში
03.8 .h და .cpp ფაილები
03.9 სავარჯიშო და შეჯამება

04 მაჩვენებლები (Pointers) 38

04.1 მეხსიერება და მისამართი
04.2 პირველი მაჩვენებელი
04.3 struct — საკუთარი ტიპი
04.4 მაჩვენებელი ობიექტზე: ისარი
04.5 nullptr — „ფენა არ არის არჩეული“
04.6 ჩამოკიდებული მაჩვენებელი
04.7 სავარჯიშო და შეჯამება

05 მითითებები (References) და const 47

05.1 მეტსახელი
05.2 მაჩვენებელი თუ მითითება?
05.3 ფუნქციაში გადაცემა: კოპირება თუ ორიგინალი
05.4 const & — უსაფრთხო დათვალიერება
05.5 QGIS-ის ხელმოწერების კითხვა
05.6 სავარჯიშო და შეჯამება

06 კლასები და ობიექტები 55

06.1 კლასი და ობიექტი
06.2 public და private
06.3 სრული მაგალითი
06.4 კონსტრუქტორი
06.5 const მეთოდი
06.6 struct და class
06.7 სავარჯიშო და შეჯამება

07 მემკვიდრეობა და პოლიმორფიზმი 61

07.1 მემკვიდრეობა
07.2 virtual და override
07.3 ტიპის გარდაქმნა: dynamic_cast
07.4 qobject_cast QGIS-ში
07.5 სავარჯიშო და შეჯამება

08 მეხსიერების მართვა და smart pointers 70

08.1 სამი ცნება: მფლობელობა, წვდომა, გადაცემა
08.2 stack და heap
08.3 new და delete
08.4 unique_ptr: ერთადერთი მფლობელი
08.5 მფლობელობის გადაცემა: std::move
08.6 shared_ptr: ბევრი მფლობელი
08.7 რა ხდება QGIS-ში
08.8 სავარჯიშო და შეჯამება

09 STL: კონტეინერები და ალგორითმები 79
09.1 vector — დინამიური მასივი
09.2 map — გასაღები → მნიშვნელობა
09.3 ალგორითმები
09.4 lambda — უსახელო ფუნქცია
09.5 ძებნა
09.6 სავარჯიშო და შეჯამება

10 Qt: QString, QVariant, signals და slots 86
10.1 QString
10.2 QList
10.3 QVariant — უნივერსალური ყუთი
10.4 QgsAttributes — ატრიბუტების ცხრილის ერთი სტრიქონი
10.5 QObject-ის ხე
10.6 signals და slots
10.7 სავარჯიშო და შეჯამება

11 api.qgis.org-ის კითხვა 95
11.1 კლასის გვერდის სქემა
11.2 ხელმოწერის გაშიფვრა
11.3 მაკროები, რომლებიც შეგხვდება
11.4 ცოცხალი მაგალითი: ამოცანიდან მეთოდამდე
11.5 ვერსიები
11.6 ბილიკი დოკუმენტაციიდან კოდამდე
11.7 ძირითადი კლასები ერთ ცხრილში
11.8 შეცდომის კითხვა
11.9 სავარჯიშო და შეჯამება

12 პროექტი: საკუთარი ინსტრუმენტი QGIS-ში 102
12.1 გეგმა
12.2 CRS და ფართობი: ჯერ ეს უნდა გავიგოთ
12.3 საფეხური 1: პოლიგონების ფართობი qgis_core-ზე
12.4 CMake: ერთი კონკრეტული გარემო
12.5 საფეხური 2: C++ პლაგინი QGIS-ის წყაროში
12.6 საფეხური 3: QGIS-ის აწყობა წყაროდან (გაფართოებული)
12.7 სავარჯიშო და შეჯამება

CHAPTER 01

01

შესავალი: რა არის QGIS-ის პროგრამირება

Introduction — what is programming inside QGIS

პროგრამირება და API · PyQGIS და C++ · QGIS-ის არქიტექტურა · კომპილაცია · პირველი C++ პროგრამა

წინამდებარე სახელმძღვანელოს მიზანია, მკითხველს დაეხმაროს QGIS-ის მომხმარებლობიდან მის პროგრამირებამდე გადასვლაში: QGIS-ის C++ API-ის დოკუმენტაციის (api.qgis.org) გააზრებასა და საკუთარი პროგრამული კოდის QGIS-ის ბირთვთან ინტეგრირების უნარის განვითარებაში.

წიგნი დაწერილია იმ მკითხველისთვის, რომელიც პროგრამირებას პირველად ეცნობა. წინასწარ საჭიროა მხოლოდ GIS-ის საბაზისო ცოდნა (ფენა, ობიექტი, კოორდინატი). Python-ისა და PyQGIS-ის ცოდნა სასარგებლოა, მაგრამ სავალდებულო არ არის: ყველა ტერმინი აქვე აიხსნება.

ვერსიები. წიგნის მაგალითები დაწერილია C++17 სტანდარტით. QGIS API-ის ხელმოწერები გადამოწმებულია `api.qgis.org`-ის 4.3.0-Master დოკუმენტაციასთან (Qt6); QGIS 3.x ვერსიებში (Qt5) ზოგი ხელმოწერა შეიძლება ოდნავ განსხვავდებოდეს. C++ მაგალითები თავები 02–09 არ საჭიროებს QGIS-ს; თავები 10–12 საჭიროებს Qt-ს და/ან QGIS-ს.

გამოცემაში განხილული ყველა პრაქტიკული მაგალითი ეფუძნება გეოგრაფიულ მონაცემებს და მოიცავს ისეთ ფუნდამენტურ კომპონენტებს, როგორებიცაა სივრცითი ფენები, გეოგრაფიული ობიექტები, კოორდინატები

და სივრცითი მახასიათებლები (მაგალითად, ფართობები).

01.1 რა არის პროგრამირება და რას ვსწავლობთ

პროგრამა არის ინსტრუქციების თანმიმდევრობა, რომელსაც კომპიუტერი ასრულებს. ინსტრუქციებს ვწერთ ტექსტად, სპეციალურ ენაზე. ამ ტექსტს **წყარო კოდი** (source code) ჰქვია, ენას კი **პროგრამირების ენა**.

> **რეცეპტი**
> წყარო კოდი რეცეპტს ჰგავს: ნაბიჯ-ნაბიჯ წერია, რა გააკეთოს შემსრულებელმა. შემსრულებელი აქ კომპიუტერია, ის კი ზუსტად ასრულებს მხოლოდ იმას, რაც დაწერილია, და არაფერს ხვდება „თავისით“.

QGIS არის GIS პროგრამა, რომელსაც ვიყენებთ რუკების სახატავად და სივრცითი ანალიზისთვის. ის თავადაც პროგრამაა და მისი კოდი ღიაა. მასთან პროგრამული გზით სამუშაოდ სამი დონე არსებობს:

დონე	ენა	რა შეგვიძლია
1. QGIS-ის Python კონსოლი და სკრიპტები	Python (PyQGIS)	გავაავტომატოთ განმეორებადი სამუშაო: ფენების დამუშავება, ექსპორტი
2. პლაგინები	Python	დავამატოთ ახალი ღილაკები და ინსტრუმენტები
3. QGIS-ის ბირთვი	C++	შევცვალოთ ან გავაფართოვოთ თავად პროგრამა, ავაგოთ სწრაფი ან დახურული კომპონენტები

ეს წიგნი მესამე დონისკენ გვატარებს: ვისწავლით C++-ს იმ მოცულობით, რაც QGIS-ის ბირთვის გასაგებად გვჭირდება.

რა არის API

API (Application Programming Interface) არის „მენიუ“ პროგრამისტისთვის: ჩამონათვალი იმისა, რა ფუნქციები და კლასები აქვს ბიბლიოთეკას და როგორ გამოვიძახოთ ისინი. QGIS-ის API-ის აღწერა ვებგვერდზეა `api.qgis.org`. ამ წიგნის მთავარი მიზანია, ეს გვერდები გავიგოთ.

01.2 PyQGIS და C++ API: ერთი ბირთვი, ორი ინტერფეისი

PyQGIS არის QGIS-ის Python-იდან გამოყენების საშუალება. ის ცალკე ბიბლიოთეკა არ არის. QGIS-ის ბირთვი C++-ზეა დაწერილი, Python-ის კლასები კი ძირითადად ავტომატურად გენერირდება (SIP-ის საშუალებით) იმავე C++ კლასებიდან. ამიტომ სახელები და ლოგიკა უმეტესად ერთი და იგივეა: `QgsVectorLayer`, `QgsFeature`, `QgsGeometry`.

> **ზუსტად**
> ორი API ერთმანეთს ძირითადად ემთხვევა, მაგრამ იდენტური არ არის: ზოგი overload, ტიპი ან მეთოდი მხოლოდ C++-შია, ზოგი კი Python-ში სხვა ფორმით ჩანს. ამიტომ ბოლო სიტყვა ყოველთვის კონკრეტული ვერსიის დოკუმენტაციას ეკუთვნის.

ბიბლიოთეკა (library) არის მზა კოდის ნაკრები, რომელსაც საკუთარ პროგრამაში ვიყენებთ, რომ ყველაფერი თავიდან არ დავწეროთ. **კლასი** (class) კი არის „ნახაზი“ რაიმე ობიექტისა (მაგალითად, ვექტორული ფენისა); კლასებს თავი 06-ში დაწვრილებით გავეცნობით.

რა არის SIP?

SIP (Python–C++ Binding Generator) არის ინსტრუმენტი, რომელიც C++-ზე დაწერილი კლასებისა და ფუნქციების Python-იდან გამოყენების შესაძლებლობას უზრუნველყოფს. QGIS-ში SIP გამოიყენება C++ კლასებსა და Python ინტერფეისს შორის დამაკავშირებელი კოდის გენერირებისთვის.

QGIS-ის ბირთვი დაწერილია C++-ზე, ხოლო PyQGIS Python-იდან ამ ბირთვის ფუნქციონალზე წვდომის საშუალებას იძლევა. SIP-ის მეშვეობით იქმნება დამაკავშირებელი ფენა, რომლის საშუალებითაც Python-ის კოდიდან

შესაძლებელია QGIS-ის C++ კლასებისა და მეთოდების გამოყენება.

მაგალითად, PyQGIS-ში შეგვიძლია გამოვიყენოთ შემდეგი კლასები:

- `QgsVectorLayer` — ვექტორული ფენის წარმოდგენა და მართვა;
- `QgsFeature` — გეოგრაფიული ობიექტის წარმოდგენა;
- `QgsGeometry` — გეომეტრიული მონაცემების წარმოდგენა და დამუშავება.

ეს კლასები QGIS-ის C++ API-ს შეესაბამება და Python-იდან მათი გამოყენება SIP-ის მიერ გენერირებული დამაკავშირებელი კოდის მეშვეობით ხდება.

მნიშვნელოვანია: SIP თავად არ არის PyQGIS და არც QGIS-ის ბირთვი. ის წარმოადგენს ერთ-ერთ ტექნოლოგიას, რომლის საშუალებითაც C++-ზე დაფუძნებული QGIS-ის ფუნქციონალი Python-იდან ხელმისაწვდომი ხდება.

> **გზამკვლევი ორ ენაზე**
> წარმოიდგინე საქართველოს რუკა ორ ენაზე. ქალაქები და გზები ერთი და იგივეა, განსხვავდება მხოლოდ წარწერების ენა. PyQGIS არის „ქართული წარწერებით“ რუკა, C++ API — „ორიგინალის“.

საბოლოოდ, QGIS-ის ძირითადი ლოგიკა (ფენა, ობიექტი, გეომეტრია) ორივე ენაში ერთია. C++-ში ახალია **გრამატიკა**: ტიპები, მაჩვენებლები, კომპილაცია. სწორედ მათ შევისწავლით.

01.3 QGIS-ის არქიტექტურა

QGIS-ის წყარო კოდი რამდენიმე ბიბლიოთეკადაა დაყოფილი:

საქაღალდე	ბიბლიოთეკა	რას შეიცავს	გეო-მაგალითი
`src/core/`	`qgis_core`	გეომეტრია, CRS, ფენები, ატრიბუტები, რენდერი	`QgsGeometry`, `QgsCoordinateReferenceSystem`
`src/gui/`	`qgis_gui`	ვიჯეტები და დიალოგები	ფენის არჩევის კომბობოქსი
`src/analysis/`	`qgis_analysis`	გეოპროცესინგი და ანალიზი	ბუფერი, გადაკვეთა
`src/app/`	`qgis`	თავად დესკტოპ პროგრამა	მენიუ, ინსტრუმენტთა პანელი

> **დაიმახსოვრე**
> `core` არ იცნობს ინტერფეისს, `gui` კი `core`-ს ეყრდნობა. ამიტომაც შეგიძლია `core` ფანჯრების გარეშე, სერვერზეც გამოიყენო.

რას ნიშნავს `core` და `gui`

- `core` — QGIS-ის ძირითადი ფუნქციონალია: გეოგრაფიული მონაცემების დამუშავება, სივრცითი ფენები, გეომეტრიები, კოორდინატთა სისტემები და სხვა.
- `gui` — გრაფიკულ ინტერფეისთან დაკავშირებული ფუნქციონალია: რუკის ფანჯარა, დიალოგური ფანჯრები, ღილაკები და მომხმარებელთან ურთიერთქმედება.

რას ნიშნავს „`core` არ იცნობს ინტერფეისს, `gui` კი `core`-ს ეყრდნობა“

ეს ნიშნავს, რომ `core`-ის ფუნქციონალი არ არის დამოკიდებული გრაფიკულ ინტერფეისზე. მას შეუძლია გეოგრაფიული მონაცემების დამუშავება ისე, რომ ეკრანზე რუკის ფანჯარა ან სხვა გრაფიკული ელემენტები საერთოდ არ არსებობდეს.

მეორე მხრივ, `gui` იყენებს `core`-ის შესაძლებლობებს, რათა ეს მონაცემები მომხმარებლისთვის ვიზუალურად წარმოადგინოს და მათთან ინტერაქცია უზრუნველყოს.

შრე	რას შეიცავს	მიმართულება
GUI	რუკის ჩვენება, ღილაკები, დიალოგები	იყენებს `core`-ს ↓
Core	ფენები, გეომეტრიები, სივრცითი გამოთვლები	არ იცნობს `gui`-ს

დამოკიდებულების მიმართულება: GUI → Core. `core`-ს GUI არ სჭირდება.

რატომ არის ეს მნიშვნელოვანი

წარმოიდგინე, რომ გჭირდება ერთი მილიონი გეოგრაფიული ობიექტის დამუშავება სერვერზე, რუკის ფანჯრის გახსნის გარეშე. `core`-ის გამოყენებით შეგიძლია:

- წაიკითხო გეოგრაფიული ფენები;
- გამოთვალო ფართობები და მანძილები;
- გაფილტრო გეოგრაფიული ობიექტები;
- დაამუშაო მონაცემები და შეინახო შედეგები.

ამ ამოცანებისთვის გრაფიკული ინტერფეისი საჭირო არ არის. მაგალითად, PyQGIS-ში:

PYTHON

```python
from qgis.core import QgsApplication, QgsVectorLayer

# QGIS-ის ინიციალიზაცია გრაფიკული ინტერფეისის გარეშე
app = QgsApplication([], False)
app.initQgis()

# ვექტორული ფენის შექმნა
layer = QgsVectorLayer(
    "/data/roads.gpkg|layername=roads",
    "roads",
    "ogr"
)

if layer.isValid():
    print("ფენა წარმატებით ჩაიტვირთა")
    print("ობიექტების რაოდენობა:", layer.featureCount())

app.exitQgis()
```

აქ `QgsApplication([], False)` მიუთითებს, რომ QGIS გრაფიკული ინტერფეისის გარეშე მუშაობს. ამას ხშირად **headless** რეჟიმს უწოდებენ.

> **შენიშვნა**
> გრაფიკული ინტერფეისის გარეშე მუშაობა არ ნიშნავს, რომ QGIS-ის ინიციალიზაცია აღარ გვჭირდება. საჭიროა გარემოს სწორად მომზადება, ხოლო კონკრეტული ოპერაციებისთვის შესაძლოა დამატებითი დამოკიდებულებებიც იყოს საჭირო.

რა უნდა დაიმახსოვრო

- `qgis.core` შეიცავს QGIS-ის ძირითად ფუნქციონალს.
- `qgis.gui` შეიცავს გრაფიკულ ინტერფეისთან დაკავშირებულ ფუნქციონალს.
- `gui` იყენებს `core`-ს, მაგრამ `core` არ არის დამოკიდებული `gui`-ზე.
- ამიტომ `core`-ის გამოყენება შესაძლებელია სერვერზე და გრაფიკული ინტერფეისის გარეშე, სათანადო ინიციალიზაციის პირობებში.

მთავარი აზრი: QGIS მხოლოდ დესკტოპ-აპლიკაცია არ არის. მისი ძირითადი გეოსივრცითი ფუნქციონალი შეიძლება გამოიყენო სერვერულ პროგრამებში, ავტომატიზაციის სკრიპტებსა და მონაცემთა დამუშავების სამუშაო პროცესებში.

01.4 ერთი და იგივე ამოცანა ორ ენაზე

ამოცანა: ფენის ყველა ობიექტის სახელის გამოტანა. ჯერ ტერმინები, რომლებსაც კოდში შევხვდებით:

ტერმინი	რას ნიშნავს
ფენა (`layer`)	მონაცემთა ერთი ნაკრები რუკაზე, მაგალითად მდინარეების ფენა
ობიექტი (`feature`)	ფენის ერთი ჩანაწერი: გეომეტრია და ატრიბუტები, მაგალითად ერთი მდინარე
ატრიბუტი	ობიექტის ერთი თვისება, მაგალითად `name` ველი
იტერატორი (`iterator`)	„მომხსენებელი“, რომელიც ფენის ობიექტებს სათითაოდ გვაწვდის

პირველად ვნახოთ იგივე ლოგიკა C++-ში. თითოეული ხაზი ცალკე აიხსნება:

C++

```
QgsVectorLayer *layer =
  qobject_cast<QgsVectorLayer *>(iface->activeLayer());   // 1
QgsFeatureIterator it = layer->getFeatures();            // 2
QgsFeature feature;                                      // 3
while (it.nextFeature(feature)) {                        // 4
  qDebug() << feature.attribute("name").toString();      // 5
}
```

1. ვიღებთ რუკაზე ახლა არჩეულ ფენას და ვაცხადებთ, რომ ის ვექტორული ფენაა (`*` და `qobject_cast` თავებში 04 და 07 აიხსნება);
2. ფენას ვთხოვთ ობიექტების იტერატორს;
3. ვქმნით ცარიელ ობიექტს, რომელშიც თითოეულს ჩავწერთ;
4. სანამ ახალი ობიექტი არსებობს (`nextFeature`), ციკლი მეორდება;
5. ვბეჭდავთ ობიექტის `name` ატრიბუტს.

თუ Python-ს იცნობ, იგივე შედეგი შედარებისთვის:

PYTHON

```
layer = iface.activeLayer()
for feature in layer.getFeatures():
    print(feature["name"])
```

C++

```
QgsVectorLayer *layer =
  qobject_cast<QgsVectorLayer *>
(iface->activeLayer());
QgsFeatureIterator it = layer-
>getFeatures();
QgsFeature feature;
while (it.nextFeature(feature)) {
  qDebug() <<
feature.attribute("name").toString(
);
}
```

ორივეგან ერთი და იგივე სახელებია (`getFeatures`, `attribute`). C++-ში ემატება ტიპები (`QgsVectorLayer`), `*` ნიშანი, `;` ყოველი ინსტრუქციის ბოლოს და ტიპის გარდაქმნა `qobject_cast`. თითოეულს ამ წიგნში ცალკე გავარჩევთ.

01.5 როგორ მუშაობს კომპილაცია

კომპიუტერი პირდაპირ ვერ კითხულობს ადამიანის დაწერილ ტექსტს. მას სჭირდება **მანქანური კოდი**, ანუ ნულებისა და ერთიანების თანმიმდევრობა.

C++ პროგრამა **კომპილდება**: წყარო კოდს **კომპილატორი** (compiler) გადააქცევს ერთ ფაილად, რომელსაც კომპიუტერი უშუალოდ ასრულებს. ამ ფაილს **შესრულებადი ფაილი** (executable) ჰქვია, Windows-ზე ის `.exe`-ა.

```
წყარო კოდი (.cpp)  →  კომპილატორი  →  შესრულებადი ფაილი  →  გაშვება
```

> **თარგმანი**
> წარმოიდგინე წიგნი, რომელიც ქართულად დაწერე. კომპილატორი პროფესიონალი მთარგმნელია: მთელ წიგნს წინასწარ თარგმნის და გიბრუნებს გამზადებულ ინგლისურ ვერსიას. ამის შემდეგ მკითხველი უკვე მზა ტექსტს კითხულობს. მთარგმნელი მეტს აღარ სჭირდება.

სინამდვილეში აწყობას ორი ნაწილი აქვს: **კომპილაცია** (თითოეული `.cpp` ფაილი ცალკე გადადის მანქანურ კოდად) და **ლინკირება** (linking), როცა ეს ნაწილები და საჭირო ბიბლიოთეკები ერთ შესრულებად ფაილად ერთდება. გაშვებისას პროგრამას შეიძლება კიდევ დასჭირდეს **დინამიური ბიბლიოთეკები** (`.dll`, `.so`), რომლებიც ცალკე ფაილებად ინახება და გაშვებისას იტვირთება.

ამიტომ პროცესს ორი ეტაპი აქვს: ჯერ **აწყობა** (build), შემდეგ **გაშვება** (run). თუ კოდში შეცდომაა, კომპილატორი აწყობისას გვეტყვის და პროგრამა საერთოდ არ შეიქმნება. ეს კარგია: შეცდომა ჩანს, სანამ ვინმე პროგრამას გამოიყენებს.

ინტერპრეტატორი
Python-ს თარგმნა არ სჭირდება მანამ, სანამ ვუშვებთ: პროგრამა სახელწოდებით **ინტერპრეტატორი** კოდს ხაზ-ხაზ კითხულობს და მაშინვე ასრულებს. ეს მოხერხებულია, თუმცა უფრო ნელია, ხოლო კოდის ტექსტი გამშვებ პირთან რჩება.

01.6 სამუშაო გარემოს მომზადება

C++ კოდის დასაწერად და გასაშვებად გჭირდება სამი რამ:

რა	რისთვის	მაგალითი
რედაქტორი	კოდის წასაკითხად და დასაწერად	Visual Studio Code, Qt Creator
კომპილატორი	კოდის გადასაყვანად	`g++` (GCC)
ტერმინალი	ბრძანებების აკრეფისთვის	PowerShell, Linux ტერმინალი

Linux-ზე (Ubuntu/Debian) კომპილატორის დასაყენებლად:

```shell
sudo apt install g++
g++ --version
```

Windows-ზე უმარტივესია MSYS2 (msys2.org): დააყენე და მის ტერმინალში გაუშვი `pacman -S mingw-w64-ucrt-x86_64-gcc`. შემდეგ `C:\msys64\ucrt64\bin` დაამატე სისტემის `Path`-ში და ახალ ტერმინალში შეამოწმე `g++ --version`. თუ ვერსია დაიბეჭდა, გარემო მზადაა.

თუ ვერ დააყენე
დასაწყისისთვის კოდის გასაშვებად შეგიძლია გამოიყენო ონლაინ კომპილატორი (მაგალითად, onlinegdb.com ან godbolt.org). თავები 02–10 არ საჭიროებს QGIS-ის დაყენებას.

01.7 პირველი C++ პროგრამა

შექმენი ფაილი `main.cpp` და ჩაწერე:

C++ main.cpp

```cpp
#include <iostream>

int main() {
    std::cout << "Hello, Georgia!" << std::endl;
    return 0;
}
```

ტერმინალში, იმავე საქაღალდეში:

SHELL

```
g++ main.cpp -o hello
./hello
```

პირველი ბრძანება აკომპილებს `main.cpp`-ს და ქმნის შესრულებად ფაილს `hello`-ს (`-o` არის „გამოსასვლელი ფაილის სახელი“). მეორე ბრძანება მას უშვებს. (Windows PowerShell-ში გაუშვი `.\hello.exe`.)

შედეგი

```
Hello, Georgia!
```

ხაზ-ხაზ

ხაზი	რას აკეთებს
`#include <iostream>`	„ჩაიტვირთე სტანდარტული ბიბლიოთეკის ნაწილი, რომელიც ეკრანზე ტექსტის გამოტანას ემსახურება“
`int main() { ... }`	პროგრამის შესასვლელი წერტილი: აქედან იწყება შესრულება. `int` ნიშნავს, რომ ფუნქცია მთელ რიცხვს დააბრუნებს
`std::cout << "..."`	ტექსტის გამოტანა ეკრანზე. `<<` „ატარებს“ ტექსტს გამოსატან არხში
`std::endl`	ახალ ხაზზე გადასვლა
`return 0;`	პროგრამა მუშაობას ამთავრებს და ატყობინებს, რომ ყველაფერი რიგზეა (`0` = წარმატება)
`;`	ყოველი ინსტრუქციის დამასრულებელი ნიშანი

> **ხშირი შეცდომები**
> პროგრამა არ აიწყობა, თუ `;` გამოგრჩა, ფიგურული ფრჩხილი `}` დაგავიწყდა ან `main`-ის სახელი შეცდომით დაწერე. კომპილატორის შეტყობინებაში ყოველთვის წაიკითხე ხაზის ნომერი: ის გიჩვენებს, სად უნდა დაიწყო ძებნა.

01.8 Python vs C++ ერთი შეხედვით

ცხრილი მათთვის, ვინც Python-ს იცნობს. თუ არ იცნობ, უბრალოდ გაიგე, რა არის C++-ის მხარე:

	Python / PyQGIS	C++ / QGIS Core
გაშვება	ინტერპრეტატორი, პირდაპირ	კომპილაცია, შემდეგ გაშვება
ტიპები	დინამიური: `x = 5`	სტატიკური: `int x = 5;`
მეხსიერება	ავტომატური	ხელით, ან smart pointer-ებით
შეცდომები	გაშვების დროს	უმეტესად კომპილაციისას
სიჩქარე	ხშირად ნელი, მოხერხებული	ხშირად მაღალი, მაგრამ დამოკიდებულია კოდზე (C++ ავტომატურად სწრაფი არ არის)
კოდის სახე	`.py` ღია ტექსტია	შედეგი ბინარულია; წყარო კოდის აღდგენა რთულია, მაგრამ შეუძლებელი არა

01.9 სავარჯიშო და შეჯამება

სავარჯიშო

1. გახსენი `api.qgis.org`, მოძებნე კლასი `QgsFeature` და ჩამოწერე სამი მეთოდის სახელი.
2. დააყენე კომპილატორი და გაუშვი `g++ --version`.
3. დაწერე და გაუშვი „Hello“ პროგრამა, ოღონდ გამოიტანე შენი ქალაქის სახელი.
4. დაამატე მეორე ხაზი: `Population: 1100000` (თბილისის მიახლოებითი მოსახლეობა).
5. განზრახ წაშალე ერთი `;` და წაიკითხე კომპილატორის შეტყობინება.

შეჯამება

- პროგრამა = ინსტრუქციები; წყარო კოდი = ამ ინსტრუქციების ტექსტი.
- API არის ბიბლიოთეკის „მენიუ“; QGIS-ის API-ის აღწერა `api.qgis.org`-ზეა.
- PyQGIS და C++ API ერთი და იგივე კლასებია სხვადასხვა ენაზე.
- `core` — ლოგიკა, `gui` — ინტერფეისი, `app` — პროგრამა.
- C++ კოდი იკომპილება: წყარო კოდი → კომპილატორი → შესრულებადი ფაილი.

CHAPTER 02

02

ტიპები და ცვლადები

Types and variables

ცვლადი · int · double · bool · auto · const · ტიპების გარდაქმნა · კოორდინატები და სიზუსტე

ამ თავში ვისწავლით, როგორ ვინახავთ მონაცემებს პროგრამაში: კოორდინატებს, ობიექტების რაოდენობას, ფენის სახელს. ეს ყველაფრის საფუძველია: QGIS-ის კოდში ყოველი ხაზი რაღაც მონაცემს ინახავს ან გადააქვს.

02.1 რა არის ცვლადი

ცვლადი (variable) არის კომპიუტერის მეხსიერებაში დაკავებული ადგილი, რომელსაც სახელი აქვს და რომელშიც მონაცემს ვინახავთ. სახელით შემდეგ ამ მონაცემს ვპოულობთ და ვცვლით.

> **ყუთი და ეტიკეტი**
> ცვლადი ეტიკეტიანი ყუთია. ეტიკეტზე წერია სახელი (`lat`), ყუთში კი რაღაც მნიშვნელობა დევს (`41.7151`). ყუთის ფორმა განსაზღვრავს, რა ჩაეტევა შიგნით: ერთში მხოლოდ მთელი რიცხვი, მეორეში ათწილადი, მესამეში ტექსტი.

C++ **მკაცრად ტიპიზებული** ენაა: ყუთის ფორმას (**ტიპს**) წინასწარ ვწერთ. ამით კომპილატორმა იცის, რამდენ მეხსიერებას დაუთმოს ცვლადს და რა მოქმედებებია დასაშვები. ტიპის გარეშე ცვლადი არ შეიქმნება.

02.2 ცვლადის შექმნა

ცვლადის შექმნას **გამოცხადება** ჰქვია. ფორმულა მარტივია:

```
ტიპი  სახელი  =  საწყისი_მნიშვნელობა;
```

C++

```
double lat = 41.7151;        // ათწილადი რიცხვი
int vertexCount = 12;        // მთელი რიცხვი
bool isValid = true;         // ჭეშმარიტი ან მცდარი
```

- `double`, `int`, `bool` — ტიპები;
- `lat`, `vertexCount`, `isValid` — ჩვენ მიერ არჩეული სახელები;
- `=` — „ჩაწერე ამ ყუთში“ (მინიჭება, არა მათემატიკური ტოლობა);
- `;` — ინსტრუქციის დასასრული;
- `//` — **კომენტარი**: ტექსტი, რომელსაც კომპილატორი უგულებელყოფს და რომელიც მხოლოდ ადამიანისთვისაა.

> **სახელდების წესები**
> სახელი შეიძლება შეიცავდეს ასოებს, ციფრებს და `_`-ს, მაგრამ ციფრით არ იწყება (`2lat` არასწორია). რეგისტრი მნიშვნელოვანია: `Lat` და `lat` სხვადასხვა ცვლადია. QGIS-ში სახელებს წერენ „აქლემის“ სტილით: `vertexCount`, `isValid`, ანუ პირველი სიტყვა პატარაა, შემდეგი სიტყვები დიდი ასოთი იწყება.

თუ Python-ს იცნობ, შედარებისთვის:

PYTHON

```python
lat = 41.7151
vertex_count = 12
is_valid = True
```

C++

```cpp
double lat = 41.7151;
int vertexCount = 12;
bool isValid = true;
```

02.3 ძირითადი ტიპები

ტიპი	რას ინახავს	გეო-მაგალითი
`int`	მთელი რიცხვი	`int vertexCount = 12;`
`double`	ათწილადი (მაღალი სიზუსტით)	`double lat = 41.7151;`
`bool`	`true` ან `false`	`bool isValid = true;`
`char`	ერთი სიმბოლო	`char hemisphere = 'N';`
`std::string`	ტექსტი	`std::string city = "Batumi";`
`qint64` (Qt-ის ტიპი)	ძალიან დიდი მთელი რიცხვი	ფენაში მილიონობით ობიექტი

`int` ჩვეულებრივ დაახლოებით ±2 მილიარდამდე რიცხვს იტევს, ამიტომ ძალიან დიდი რაოდენობებისთვის (ობიექტების რაოდენობა დიდ ფენაში) QGIS იყენებს `qint64`-ს. თუ რიცხვი ტიპის დიაპაზონს გადააჭარბებს, შედეგი არასწორი იქნება (**გადავსება**, overflow). კონტეინერის ზომას სხვა ტიპი აბრუნებს, `std::size_t` (ან `size_t`): ის არაუარყოფითი მთელი რიცხვია, ზომების და ინდექსებისთვის განკუთვნილი (თავი 05-ში შეგხვდება). ტექსტის ტიპს, რომელსაც QGIS-ში სინამდვილეში ვხვდებით (`QString`), თავი 10-ში გავეცნობით.

> **კოორდინატებისთვის**
> კოორდინატებს ყოველთვის `double`-ში ვინახავთ. QGIS-ის `QgsPointXY` შიგნით სწორედ ორ `double`-ს შეიცავს.

02.4 კოდი შედეგით

ახლა ცვლადებს ეკრანზე გამოვიტანთ. `std::cout` არის გამოტანის არხი (ქვეთავი 1.7-დან), ხოლო `<<` ჯაჭვივით აერთებს ერთმანეთზე გამოსატან ნაწილებს. `"\n"` ნიშნავს „ახალი ხაზი“.

C++ coords.cpp

```cpp
#include <iostream>

int main() {
    double lat = 41.7151;
    double lon = 44.8271;
    int vertexCount = 12;
    bool isValid = true;

    std::cout << "Tbilisi: " << lat << ", " << lon << "\n";
    std::cout << "Vertices: " << vertexCount << "\n";
    std::cout << std::boolalpha << "Valid: " << isValid << "\n";
    return 0;
}
```

შედეგი

```
Tbilisi: 41.7151, 44.8271
Vertices: 12
Valid: true
```

`std::boolalpha` კომპიუტერს ეუბნება: `bool` გამოიტანე სიტყვით (`true` / `false`). მის გარეშე დაიბეჭდებოდა `1` ან `0`.

02.5 const და auto

`const` (constant, „მუდმივა“) ნიშნავს, რომ მნიშვნელობას შემდეგ აღარ შევცვლით. `auto` კომპილატორს ეუბნება, რომ ტიპი თვითონ გამოიცნოს მარჯვენა მხარიდან.

C++

```cpp
const double EARTH_RADIUS_KM = 6371.0;   // დედამიწის რადიუსი, არ იცვლება
auto area = 50.0;                        // კომპილატორი ხვდება: double
auto count = 12;                         // კომპილატორი ხვდება: int
```

> **ყურადღება**
> `const` ცვლადს მნიშვნელობა მხოლოდ შექმნისას ეძლევა. მოგვიანებით ცვლილების მცდელობა კომპილაციის შეცდომას იძლევა. ეს კარგია: შეცდომა კოდის გაშვებამდე ჩანს. QGIS-ის კოდში `const` ძალიან ხშირად გვხვდება (იხ. თავი 05).

02.6 მთელი რიცხვების გაყოფა

ეს პირველი ხაფანგია: როცა ორივე რიცხვი `int`-ია, გაყოფის შედეგიც `int`-ია და წილადი ნაწილი უბრალოდ იკვეთება.

C++ `division.cpp`

```cpp
#include <iostream>

int main() {
    int totalHectares = 7;
    int parcels = 2;

    std::cout << totalHectares / parcels << "\n";
    std::cout << totalHectares / 2.0 << "\n";
    return 0;
}
```

შედეგი

```
3
3.5
```

ჯერ 7 ჰექტარი გავყავით 2 ნაკვეთზე და მივიღეთ `3` (არა `3.5`), რადგან ორივე რიცხვი მთელია. მეორე ხაზში `2.0` ათწილადია, ამიტომ მთელი გამოთვლა ათწილადებში ხდება და შედეგია `3.5`.

> **ფესი**
> საშუალოს ან ფართობის გამოთვლისას დარწმუნდი, რომ ერთი მაინც `double`-ია, თორემ შედეგი „მოიჭრება“.

02.7 სიზუსტე და გარდაქმნა

კომპიუტერი ათწილადებს ორობით სისტემაში ინახავს და ზოგიერთ რიცხვს ზუსტად ვერ წარმოადგენს. ეს ისეთივე პრობლემაა, როგორც ათობით სისტემაში 1/3 = 0.3333…: უსასრულოა და სადმე უნდა შევწყვიტოთ. ამიტომ ორი ათწილადის `==`-ით შედარება სანდო არ არის.

C++ `precision.cpp`

```cpp
#include <iostream>
#include <cmath>

int main() {
    double a = 0.1 + 0.2;
    std::cout << std::boolalpha;
    std::cout << (a == 0.3) << "\n";
    std::cout << (std::abs(a - 0.3) < 1e-9) << "\n";

    int whole = static_cast<int>(3.99);
    std::cout << whole << "\n";
    return 0;
}
```

შედეგი

```
false
true
3
```

ახსნა:

- `a == 0.3` სამართლიანად გვეპასუხება `false`, რადგან `0.1 + 0.2` კომპიუტერში ზუსტად `0.3` არ არის;
- `std::abs(a - 0.3) < 1e-9` ამოწმებს, მათ შორის სხვაობა 0.000000001-ზე ნაკლებია თუ არა (`std::abs` აბსოლუტური მნიშვნელობაა, `1e-9` კი 10^{-9}) და პასუხია `true`;
- `static_cast<int>(3.99)` არის **აშკარა გარდაქმნა** ათწილადიდან მთელზე და ათწილადი ნაწილი უბრალოდ იკვეთება: `3`.

> **წესი**
> ათწილადები ვადარო სიზუსტით (`abs(a - b) < epsilon`). გარდაქმნა გააკეთე აშკარად, `static_cast<...>(...)`-ით, რომ ყველამ დაინახოს, სად იკარგება ინფორმაცია.

> **`1E-9` უნივერსალური არ არის**
> ზღვარი ამოცანაზეა დამოკიდებული. გრადუსებში მოცემული კოორდინატებისთვის `1e-9` ძალიან მკაცრია (დაახლოებით მილიმეტრის მეასედი), მეტრებში მოცემული ათასობით მეტრის სიდიდისთვის კი სხვა სიზუსტეა საჭირო. აირჩიე ზღვარი იმის მიხედვით, რა სიზუსტეა შენი მონაცემისთვის აზრიანი.

გარდაქმნების სამი სახე გაარჩიე: (1) **აშკარა გარდაქმნა** (`static_cast`), როცა შენ აცხადებ, რომ ტიპი იცვლება; (2) **სიზუსტის დაკარგვა**, როცა ათწილადი ნაწილი იკვეთება ან დიდი რიცხვი პატარა ტიპში ეტევა მხოლოდ დაახლოებით (`double` → `int`, `double` → `float`); (3) **დიაპაზონის გადაჭარბება**, როცა მნიშვნელობა ტიპში საერთოდ არ ეტევა.

02.8 სავარჯიშო და შეჯამება

სავარჯიშო

1. გამოაცხადე ცვლადები ბათუმისთვის: განედი `41.6168`, გრძედი `41.6367`, მოსახლეობა `int`-ად (მიახლოებით 170000). გამოიტანე ისინი.
2. წარმოიდგინე 5 ნაკვეთი, საერთო ფართობი 12 ჰექტარი. გამოთვალე საშუალო ფართობი სწორად (`double`) და არასწორად (`int`-ებით). შეადარე.
3. რას დაბეჭდავს `static_cast<int>(-2.7)`? ჯერ გამოიცანი, მერე შეამოწმე.
4. შეეცადე `const double PI = 3.14;`-ს მოგვიანებით შეცვლას და წაიკითხე კომპილატორის შეტყობინება.

შეჯამება

- ცვლადი = სახელიანი ყუთი მეხსიერებაში; ტიპი = ყუთის ფორმა.
- ძირითადი ტიპები: `int`, `double`, `bool`, `char`, `std::string`.
- კოორდინატები — `double`, რაოდენობები — `int` ან `qint64`.
- `int / int` იძლევა `int`-ს.
- ათწილადებს ვადარებთ ზღვრით, არა `==`-ით.
- `const` = არ იცვლება; `auto` = ტიპს თვითონ გამოიცნობს.

CHAPTER 03

03

ფუნქციები და namespace

Functions and namespaces

რა არის ფუნქცია · პარამეტრები და დაბრუნება · overload · default არგუმენტი · .h/.cpp დაყოფა · namespace და ::

პროგრამა დიდია და ერთსა და იმავე გამოთვლას ბევრგან ვიყენებთ. თუ ყოველ ჯერზე თავიდან ვწერთ, კოდი იზრდება და შეცდომებიც მატულობს. ამიტომ განმეორებადი ნაწილი ერთხელ ვწერთ და სახელს ვარქმევთ. ასეთ დასახელებულ ნაწილს **ფუნქცია** ჰქვია.

03.1 რა არის ფუნქცია

> **ხელსაწყო**
> ფუნქცია არის ხელსაწყო, რომელსაც რაღაცას აწვდი (შესასვლელი) და რაღაცას გიბრუნებს (გამოსასვლელი). ფართობის გამოთვლის ხელსაწყოს სიგანესა და სიგრძეს აძლევ და ფართობს იღებ. შიგნით როგორ მუშაობს, გამოძახებისას არ გვაინტერესებს.

ფუნქციის დაწერა ორი მიზნით გვჭირდება: ერთი და იგივე კოდი ბევრგან არ გავიმეოროთ და პროგრამა დავყოთ ადვილად გასაგებ ნაწილებად. ბრძანება `main()`, რომელსაც ხშირად ვხვდებით, ასევე ფუნქციაა, პროგრამის მთავარი ფუნქცია.

03.2 ფუნქციის ანატომია

```C++
double calculateArea(double width, double height) {
    return width * height;
}
```

ნაწილი	მაგალითში	მნიშვნელობა
დაბრუნების ტიპი	`double`	რა სახის მნიშვნელობას დაგიბრუნებს
სახელი	`calculateArea`	როგორ გამოვიძახოთ
პარამეტრები	`double width, double height`	რა უნდა გადავცეთ (თითოეულს ტიპი აქვს)
სხეული	`{ ... }`	რას აკეთებს ფუნქცია
`return`	`return width * height;`	უბრუნებს შედეგს გამომძახებელს და ამთავრებს ფუნქციას

გამოძახება ასე ხდება: `calculateArea(5.0, 10.0)`. ფრჩხილებში გადაცემულ მნიშვნელობებს **არგუმენტები** ეწოდება, ფუნქციის ხელმოწერაში მათ **პარამეტრები** ჰქვია. თუ ფუნქცია არაფერს აბრუნებს, დაბრუნების ტიპია `void` („არაფერი“).

> **გავრცელებული შეცდომა**
> ფუნქციის განსაზღვრისას ფრჩხილების შემდეგ `;` არ დაიდება: `double f(double a); { ... }` არასწორია. `;` ფუნქციას „დეკლარაციად“ აქცევს და ფიგურული ფრჩხილები ფუნქციას აღარ ეკუთვნის.

03.3 namespace — სახელების საქაღალდე

წარმოიდგინე დიდი პროექტი, სადაც ორ სხვადასხვა ბიბლიოთეკას ორივეს აქვს ფუნქცია `calculateArea`. კომპილატორმა როგორ უნდა გაარჩიოს? ამ პრობლემას **namespace** ხსნის.

საქაღალდეები კომპიუტერში
ორ საქაღალდეში (`ProjectA`, `ProjectB`) შეიძლება ერთი სახელის ფაილი გვქონდეს, `data.txt`. კონფლიქტი არ იქნება, რადგან მისამართი განსხვავებულია. `namespace` სწორედ ასეთი საქაღალდეა კოდისთვის.

C++ area.cpp

```cpp
#include <iostream>

namespace GIS {
    double calculateArea(double width, double height) {
        return width * height;
    }
}

int main() {
    double area = GIS::calculateArea(5.0, 10.0);
    std::cout << "Area: " << area << std::endl;
    return 0;
}
```

შედეგი

```
Area: 50
```

აქ ფუნქცია `GIS` namespace-შია მოთავსებული. მისი გამოძახებისას ვწერთ `GIS::calculateArea(...)`.

`::` სახელდება **scope resolution operator** და ნიშნავს „შედი ამ namespace-ში და იქიდან აიღე“. იგივე იდეაა `std::cout`-ში, რომელსაც ისედაც ვიყენებდით: `std` არის C++-ის სტანდარტული ბიბლიოთეკის namespace-ის სახელი (standard), `cout` კი მასში მოთავსებული ობიექტი.

> **QGIS-ში**
> `Qgis::WkbType`, `Qgis::DistanceUnit` — ეს `Qgis` namespace-ია. კლასებს კი `Qgs` პრეფიქსი აქვთ: `QgsFeature`, `QgsProject`. პრეფიქსიც ერთგვარი namespace-ია ძველი სტილით.

03.4 overload — ერთი სახელი, სხვადასხვა პარამეტრები

C++-ში ერთსა და იმავე სახელზე რამდენიმე ფუნქცია შეიძლება არსებობდეს, თუ პარამეტრები განსხვავდება. ამას **overload** (გადატვირთვა) ჰქვია. კომპილატორი თავად ირჩევს სწორს არგუმენტების რაოდენობისა და ტიპის მიხედვით.

C++ overload.cpp

```cpp
#include <iostream>

namespace GIS {
    double calculateArea(double width, double height) {
        return width * height;                  // მართკუთხედი
    }
    double calculateArea(double radius) {
        return 3.14159 * radius * radius;       // წრე
    }
}

int main() {
    std::cout << GIS::calculateArea(5.0, 10.0) << "\n";
    std::cout << GIS::calculateArea(10.0) << "\n";
    return 0;
}
```

შედეგი

```
50
314.159
```

პირველი გამოძახება ორი არგუმენტითაა და მართკუთხედის ვერსიას იღებს, მეორე ერთი არგუმენტით და წრის ვერსიას. QGIS-ის API სავსეა ასეთი overload-ებით: მაგალითად, `QgsPointXY` რამდენიმე სხვადასხვა გზით იქმნება.

03.5 default პარამეტრი

პარამეტრს შეიძლება ჰქონდეს ნაგულისხმევი მნიშვნელობა. თუ არგუმენტს არ გადავცემთ, ის გამოიყენება.

C++ units.cpp

```cpp
#include <iostream>

double toMeters(double value, double factor = 1.0) {
    return value * factor;
}

int main() {
    std::cout << toMeters(2.5, 1000.0) << "\n";   // კილომეტრი -> მეტრი
    std::cout << toMeters(7.0) << "\n";            // უკვე მეტრებშია
    return 0;
}
```

შედეგი

```
2500
7
```

პირველ გამოძახებაში ვაძლევთ 2.5 კმ-ს და გადამყვან კოეფიციენტს 1000. მეორეში კოეფიციენტი არ გადაგვიცია და გამოიყენა `1.0`. default მნიშვნელობა მხოლოდ ბოლო პარამეტრებს შეიძლება ჰქონდეთ. ეს QGIS-ში ძალიან

გავრცელებულია, მაგალითად `getFeatures()` პარამეტრის გარეშეც გამოიძახება.

03.6 მანძილი ორ წერტილს შორის

C++ distance.cpp

```cpp
#include <iostream>
#include <cmath>

namespace GIS {
    double distance(double x1, double y1, double x2, double y2) {
        return std::hypot(x2 - x1, y2 - y1);
    }
}

int main() {
    std::cout << GIS::distance(0.0, 0.0, 3.0, 4.0) << "\n";
    return 0;
}
```

შედეგი

```
5
```

`std::hypot(a, b)` არის `√(a² + b²)`, ანუ პითაგორას თეორემა: მანძილი გეგმურ (დეკარტულ) კოორდინატებში.

მხოლოდ ბრტყელი კოორდინატებისთვის
ეს ფუნქცია სწორია, როცა კოორდინატები ბრტყელ სიბრტყეზეა (მაგალითად, მეტრებში, UTM-ში). გეოგრაფიულ კოორდინატებში (განედი/გრძედი გრადუსებით) ასეთი ევკლიდური მანძილი რეალურ მანძილს არ იძლევა, რადგან გრადუსი სხვადასხვა განედზე სხვადასხვა სიგრძისაა. ასეთ შემთხვევაში QGIS-ში `QgsDistanceArea`-ს ვიყენებთ (თავი 12). ::: წერტილები (0,0) და (3,4) სამკუთხედის გვერდებია 3 და 4, ჰიპოტენუზა კი 5. `<cmath>` არის მათემატიკური ფუნქციების ბიბლიოთეკა.

03.7 რამდენიმე ფუნქცია ერთ ფაილში

ახლა ყველაფერი ერთად: ერთი namespace, სამი ფუნქცია, რომლებიც ერთ ნაკვეთს ეხება.

C++ parcel_tools.cpp

```cpp
#include <cmath>
#include <iostream>

namespace GIS {
    double area(double width, double height) {
        return width * height;
    }
    double perimeter(double width, double height) {
        return 2.0 * (width + height);
    }
    double distance(double x1, double y1, double x2, double y2) {
        return std::hypot(x2 - x1, y2 - y1);
    }
}

int main() {
    std::cout << "Area: " << GIS::area(20.0, 15.0) << "\n";
    std::cout << "Perimeter: " << GIS::perimeter(20.0, 15.0) << "\n";
    std::cout << "Diagonal: " << GIS::distance(0.0, 0.0, 20.0, 15.0) << "\n";
    return 0;
}
```

შედეგი

```
Area: 300
Perimeter: 70
Diagonal: 25
```

20 × 15 მ ნაკვეთის დიაგონალია √(20² + 15²) = √625 = 25 მ.

03.8 .h და .cpp ფაილები

დიდ პროექტში ფუნქციის **დეკლარაცია** (რა ჰქვია, რა აქვს შესასვლელში და გამოსასვლელში) და **განსაზღვრება** (როგორ მუშაობს) ცალკე ფაილებშია:

- `.h` **(header) ფაილი** — დეკლარაცია, ერთგვარი „სარჩევი“; ის კომპილატორს ეუბნება, რომ ასეთი ფუნქცია არსებობს და როგორი ტიპები აქვს; მას სხვა ფაილები იყენებენ;
- `.cpp` **ფაილი** — განსაზღვრება, თავად კოდი.

ეს გაყოფა ფაილების მექანიკური დაყოფა კი არა, **ინტერფეისისა და რეალიზაციის გამიჯვნაა**: გამომძახებელს მხოლოდ ინტერფეისი (რა აკეთებს) სჭირდება, შიგნით როგორ მუშაობს, მისი საქმე არ არის.

QGIS-ის მთელი წყარო კოდი ასეა აგებული, `api.qgis.org` კი სწორედ `.h` ფაილებიდან გენერირდება. ამიტომაც ამ ფაილებს ხშირად წაიკითხავ.

C++ gis_utils.h

```cpp
#pragma once

namespace GIS {
    double distance(double x1, double y1, double x2, double y2);
}
```

C++ gis_utils.cpp

```cpp
#include "gis_utils.h"
#include <cmath>

double GIS::distance(double x1, double y1, double x2, double y2) {
    return std::hypot(x2 - x1, y2 - y1);
}
```

C++ main.cpp

```cpp
#include <iostream>
#include "gis_utils.h"

int main() {
    std::cout << GIS::distance(0.0, 0.0, 3.0, 4.0) << "\n";
}
```

ორივე `.cpp` ფაილი ერთად უნდა დაკომპილდეს:

```shell
g++ main.cpp gis_utils.cpp -o gis
```

შენიშვნა

`#include <...>` — სისტემური ბიბლიოთეკა, `#include "..."` — შენი საკუთარი ფაილი. `#pragma once` იცავს header ფაილს ორჯერ ჩართვისგან.

03.9 სავარჯიშო და შეჯამება

სავარჯიშო

1. დაწერე `GIS::perimeter(double width, double height)` (მართკუთხა ნაკვეთის პერიმეტრი) და შეამოწმე 5 × 10 ზომაზე (უნდა მიიღო 30).
2. დაამატე overload კვადრატისთვის: `perimeter(double side)`.
3. გამოთვალე მანძილი წერტილებს შორის (2, 3) და (6, 6) (უნდა მიიღო 5).
4. გადაიტანე ფუნქციები `gis_utils.h` / `gis_utils.cpp` ფაილებში და გაუშვი.

შეჯამება

- ფუნქცია = ერთხელ დაწერილი, სახელდებული ინსტრუქციათა ნაკრები; ის იღებს არგუმენტებს და აბრუნებს შედეგს.
- ფუნქცია = დაბრუნების ტიპი + სახელი + ტიპიზებული პარამეტრები.
- `namespace` იცავს სახელებს კონფლიქტისგან, `::` მასში შედის.
- overload და default პარამეტრები QGIS API-ში ყველგანაა.
- `.h` — დეკლარაცია, `.cpp` — განსაზღვრება.

CHAPTER 04

04

მაჩვენებლები (Pointers)

Pointers

მეხსიერება და მისამართი · struct · & და * · -> · nullptr · ჩამოკიდებული მაჩვენებელი

ეს ყველაზე მნიშვნელოვანი თემაა. QGIS-ის API-ში ყოველ მეორე ფუნქციას აქვს `*` ან `->`: `QgsMapLayer *layer`, `layer->name()`. ამ თავის შემდეგ ისინი გასაგები იქნება.

04.1 მეხსიერება და მისამართი

როცა ცვლადს ვქმნით, კომპიუტერი მეხსიერებაში (RAM) ადგილს გვიმზადებს. წარმოიდგინე მეხსიერება უზარმაზარ საწყობად, სადაც უამრავი ყუთია და თითოეულს **მისამართი** (ნომერი) აქვს. ცვლადის მნიშვნელობა ერთ-ერთ ამ ყუთში დევს.

მანამდე ცვლადს მხოლოდ სახელით მივმართავდით (`layerCount`). C++-ს შეუძლია გვითხრას მისი **მისამართიც**, და სწორედ ამაზეა აგებული **მაჩვენებელი** (pointer): ცვლადი, რომელიც მნიშვნელობას კი არა, სხვა ცვლადის მისამართს ინახავს.

> **კურიერი**
> კურიერს მძიმე ამანათი მოუვიდა. ის მას ადრესატთან კი არ მიაქვს, არამედ ეუბნება: „ამანათი ამ საწყობის ამ მისამართზეა“. მაჩვენებელი არის სწორედ **მისამართი**, არა თავად ამანათი. ბევრად იაფია მისამართის გადაცემა, ვიდრე მილიონობით წერტილიანი ფენის კოპირება.

ეს QGIS-ისთვის არსებითია: ფენა შეიძლება უზარმაზარი იყოს, ამიტომ ფუნქციებს ხშირად თავად ფენას კი არა, მისი მისამართის გადავცემთ.

ორი სიმბოლო:

სიმბოლო	სახელი	რას აკეთებს
`&x`	address-of	აბრუნებს `x`-ის მისამართს
`*p`	dereference	გადადის მისამართზე და კითხულობს მნიშვნელობას

ტიპის გვერდით `*` ნიშნავს „ეს მაჩვენებელია“: `int *ptr` ნიშნავს „მაჩვენებელი `int`-ზე“.

04.2 პირველი მაჩვენებელი

C++ ptr.cpp

```cpp
#include <iostream>

int main() {
    int layerCount = 5;
    int *ptr = &layerCount;

    std::cout << "Layer count: " << layerCount << "\n";
    std::cout << "Address: " << ptr << "\n";
    std::cout << "Value at address: " << *ptr << "\n";
    return 0;
}
```

შედეგი

```
Layer count: 5
Address: 0x7ffd5c3a1a24
Value at address: 5
```

გავარჩიოთ:

- `int layerCount = 5;` — ჩვეულებრივი ცვლადი;
- `int *ptr = &layerCount;` — `ptr` არის მაჩვენებელი და მასში ჩაიწერა `layerCount`-ის მისამართი;
- `ptr` დაბეჭდავს მისამართს, `*ptr` კი „გადადის ამ მისამართზე და ამოიღებს მნიშვნელობას“, ანუ `5`-ს.

> **შენიშვნა**
> მისამართი ყოველ გაშვებაზე და ყოველ კომპიუტერზე განსხვავებულია (Windows-ზე ის ასე გამოიყურება: `000000BF7C3BF970`). მნიშვნელობა კი ერთი და იგივეა.

სამი ხაზი = სამი ფაქტი: **რა ძევს** (`5`), **სად ძევს** (მისამართი), **რას ვპოულობთ იქ** (`*ptr`).

04.3 struct — საკუთარი ტიპი

ბევრი რამ ერთ ცვლადში არ ეტევა. ფენას აქვს სახელიც და ობიექტების რაოდენობაც. ასეთ შემთხვევაში ვქმნით **struct**-ს: ტიპს, რომელიც რამდენიმე ცვლადს აერთიანებს.

```C++
struct Layer {
    std::string name;       // ფენის სახელი (ტექსტი)
    int featureCount;       // ობიექტების რაოდენობა
};

Layer rivers{"Rivers", 42};     // ახალი ცვლადი: სახელი Rivers, 42 ობიექტი
std::cout << rivers.name;       // წერტილით ვუკავშირდებით წევრებს
```

`name` და `featureCount` არის `Layer`-ის **წევრები**. ობიექტის წევრთან წერტილით მივდივართ: `rivers.name`. struct-ის უფრო ძლიერ ვერსიას (**class**) თავი 06-ში გავეცნობით.

04.4 მაჩვენებელი ობიექტზე: ისარი

მაჩვენებელი ხშირად მიუთითებს მთელ ობიექტზე. მაშინ წევრთან მისასვლელად წერტილის ნაცვლად `->` (ისარი) გამოიყენება.

C++ layer_ptr.cpp

```cpp
#include <iostream>
#include <string>

struct Layer {
    std::string name;
    int featureCount;
};

int main() {
    Layer rivers{"Rivers", 42};
    Layer *ptr = &rivers;

    std::cout << ptr->name << ": " << ptr->featureCount << "\n";

    ptr->featureCount += 8;                       // იგივეა, რაც
(*ptr).featureCount += 8
    std::cout << rivers.featureCount << "\n";
    return 0;
}
```

შედეგი

```
Rivers: 42
50
```

ხედავ? `ptr`-ით ცვლილება პირდაპირ `rivers`-ს შეეხო, რადგან ორივე ერთსა და იმავე ობიექტზე მიუთითებს, კოპირება არ მომხდარა. `+=` ნიშნავს „დაუმატე“: `featureCount += 8` იგივეა, რაც `featureCount = featureCount + 8`.

> **წესი**
> ობიექტზე — `.` (`rivers.name`), მაჩვენებლით — `->` (`ptr->name`). ზუსტად ამიტომ გვაქვს QGIS-ში `layer->featureCount()`.

04.5 nullptr — „ფენა არ არის არჩეული“

მაჩვენებელი შეიძლება არაფერზე მიუთითებდეს. ამას `nullptr` ჰქვია და ნიშნავს „ცარიელი მისამართი“.

C++ nullptr.cpp

```cpp
#include <iostream>
#include <string>

struct Layer {
    std::string name;
    int featureCount;
};

int main() {
    Layer *activeLayer = nullptr;

    if (activeLayer == nullptr) {
        std::cout << "No active layer\n";
    } else {
        std::cout << activeLayer->name << "\n";
    }
    return 0;
}
```

შედეგი

```
No active layer
```

ეს QGIS-ის ყოველდღიური სიტუაციაა: ფუნქცია `iface->activeLayer()` აბრუნებს `nullptr`-ს, თუ ფენა არ არის მონიშნული. `if (...) {...} else {...}` არის პირობის შემოწმება: თუ პირობა სრულდება, სრულდება პირველი ბლოკი, წინააღმდეგ შემთხვევაში მეორე.

თუ Python-ს იცნობ, `nullptr` მისი `None`-ის ანალოგია:

PYTHON

```python
layer = iface.activeLayer()
if layer is None:
    print("No active layer")
```

C++

```cpp
QgsMapLayer *layer = iface-
>activeLayer();
if (layer == nullptr) {
  qDebug() << "No active layer";
}
```

ხშირი შეცდომა

`nullptr`-ზე `->` გამოძახება პროგრამას ამსხვრევს (segmentation fault / crash). მაჩვენებელი ყოველთვის უნდა შეამოწმო `nullptr`-ზე, სანამ მას გამოიყენებ.

ორი სხვადასხვა შემოწმება

`ptr != nullptr` ნიშნავს მხოლოდ, რომ მაჩვენებელს რაღაც მისამართი უწერია. ის **არ გარანტირებს**, რომ იმ მისამართზე ობიექტი ჯერ კიდევ არსებობს (იხ. ჩამოკიდებული მაჩვენებელი ქვემოთ). ეს ორი რამ ერთმანეთისგან განსხვავდება: „მაჩვენებელი ცარიელი არ არის“ და „ობიექტი ცოცხალია“. მეორეს მხოლოდ ობიექტის სიცოცხლისა და მფლობელობის წესები განსაზღვრავს (თავი 08).

QGIS-ში ფენის ტიპის გასარკვევად გამოიყენება `qobject_cast`: Qt-ის ობიექტებისთვის განკუთვნილი უსაფრთხო გარდაქმნა, რომელიც `nullptr`-ს აბრუნებს, თუ ობიექტი მოთხოვნილი ტიპის არ არის. მას თავი 07-ში `dynamic_cast`-თან შევადარებთ.

04.6 ჩამოკიდებული მაჩვენებელი

საშიში ადგილი

თუ ობიექტი აღარ არსებობს, მაჩვენებელი მაინც ინახავს ძველ მისამართს. ეს ჰგავს კურიერს, რომელიც მისამართზე მიდის, სადაც სახლი უკვე დაანგრიეს.

C++ dangling.cpp

```cpp
int *makeCounter() {
    int count = 10;
    return &count;      // შეცდომა: count ფუნქციის დასრულებისას იშლება
}
```

`count` **ლოკალური ცვლადია**: ის მხოლოდ ფუნქციის შიგნით არსებობს და მისი დასრულებისას იშლება. დაბრუნებული მისამართი უკვე არავის ეკუთვნის. ასეთ მაჩვენებელს ჩამოკიდებული (dangling) ჰქვია. მაჩვენებლის გამოყენება უსაფრთხოა მხოლოდ მანამ, სანამ ობიექტი ცოცხალია, ანუ ობიექტის **სიცოცხლის ციკლი** და **მფლობელობა** (ვინ არის პასუხისმგებელი მის წაშლაზე) განსაზღვრავს, როდის შეიძლება მაჩვენებლის გამოყენება. ამ საკითხს თავი 08 ეძღვნება.

04.7 სავარჯიშო და შეჯამება

სავარჯიშო

1. შექმენი `struct Parcel { std::string owner; double area; };`. გააკეთე მაჩვენებელი და `->`-ით გამოიტანე მფლობელი და ფართობი.
2. ფართობი გაზარდე 100 კვ.მ-ით მაჩვენებლის საშუალებით და შეამოწმე თავდაპირველ ობიექტში.
3. შექმენი `Parcel *p = nullptr;` და დაამატე შემოწმება, რომ პროგრამა არ დაიმსხვრეს.

შეჯამება

- მეხსიერების ყოველ ადგილს მისამართი აქვს.
- მაჩვენებელი ინახავს **მისამართს**; `&` მისამართს იღებს, `*` მნიშვნელობას კითხულობს.
- `struct` რამდენიმე ცვლადს ერთ ტიპად აერთიანებს; წევრებთან წერტილით ვუკავშირდებით.
- ობიექტზე მაჩვენებლით ვიყენებთ `->`-ს.
- `nullptr` = „არაფერზე მიუთითებს“; ყოველთვის შეამოწმე გამოყენებამდე.
- ლოკალური ცვლადის მისამართს ფუნქციიდან არ აბრუნებენ.

CHAPTER 05

05

მითითებები (References) და const

References and const

& როგორც მეტსახელი · გადაცემა მნიშვნელობით და მითითებით · const & · QGIS ხელმოწერების კითხვა

წინა თავში მაჩვენებელი გავიგეთ. QGIS-ის API-ში მასთან ერთად ყველგან გვხვდება `&`: `const QgsFeature &feature`. ამ თავის შემდეგ ამ ჩანაწერს დაუფიქრებლად წაიკითხავ.

ჯერ ერთი მნიშვნელოვანი ცნება: **ასლი** (copy). როცა ფუნქციას ობიექტს ვაძლევთ, C++ ნაგულისხმევად ობიექტის ასლს ქმნის და ფუნქცია ასლზე მუშაობს. ორიგინალი არ იცვლება, მაგრამ ასლის შექმნა დიდი ფენისთვის ნელი და მეხსიერებატევადია. მითითება ამ ორივე პრობლემას გვარჩენს.

05.1 მეტსახელი

მითითება (reference) არის ცვლადის **მეტსახელი**: ერთი ობიექტი, ორი სახელი.

C++ alias.cpp

```cpp
#include <iostream>

int main() {
    int featureCount = 100;
    int &ref = featureCount;     // ref არის featureCount-ის მეტსახელი

    ref = 300;
    std::cout << featureCount << "\n";
    return 0;
}
```

შედეგი

```
300
```

`int &ref = featureCount;` ნიშნავს: „`ref` არის `featureCount`-ის მეორე სახელი“. შემდეგ `ref = 300` რეალურად `featureCount`-ს ცვლის. აქ `&` ტიპის გვერდით წერია და ნიშნავს „მითითება“; წინა თავში `&x` (ცვლადის წინ) ნიშნავდა „მისამართი“. ორივე ერთი სიმბოლოა, მაგრამ მნიშვნელობა მდებარეობით განისაზღვრება.

> **ბანკის პერსონალური ასისტენტი**
> სოლო მომსახურების ასისტენტი ერთხელ და სამუდამოდ გიმაგრდება. ის არ იცვლება ყოველ ვიზიტზე და ცარიელი არ არის. მითითება ზუსტად ასეა: იბადება კონკრეტულ ობიექტზე მიბმული, ვეღარ გადაიბმება სხვაზე და `nullptr` ვერ იქნება.

05.2 მაჩვენებელი თუ მითითება?

	მაჩვენებელი `*`	მითითება `&`
შეიძლება ცარიელი იყოს	დიახ, `nullptr`	არა
შეიძლება სხვაზე გადაიბას	დიახ	არა
წევრთან მისვლა	`p->name`	`r.name`
როდის	ობიექტი შეიძლება არ არსებობდეს	ობიექტი აუცილებლად არსებობს

მოკლედ

მაჩვენებელი — როცა ობიექტი შეიძლება არ არსებობდეს ან შეიცვალოს.
მითითება — როცა ობიექტი უეჭველად არსებობს.

05.3 ფუნქციაში გადაცემა: კოპირება თუ

ორიგინალი

C++ byref.cpp

```cpp
#include <iostream>
#include <string>

struct Layer {
    std::string name;
    int featureCount;
};

void addByValue(Layer layer, int n) {
    layer.featureCount += n;          // ცვლის ასლს
}

void addByRef(Layer &layer, int n) {
    layer.featureCount += n;          // ცვლის ორიგინალს
}

int main() {
    Layer roads{"Roads", 100};

    addByValue(roads, 25);
    std::cout << "After value: " << roads.featureCount << "\n";

    addByRef(roads, 25);
    std::cout << "After ref: " << roads.featureCount << "\n";
    return 0;
}
```

შედეგი

```
After value: 100
After ref: 125
```

პირველ შემთხვევაში ფუნქციამ ობიექტის **ასლი** მიიღო და ორიგინალი არ შეცვლილა. მეორეში `&`-ის გამო იმუშავა თავად ორიგინალზე. ტერმინები: პირველს **გადაცემა მნიშვნელობით** (pass by value) ჰქვია, მეორეს **გადაცემა მითითებით** (pass by reference).

05.4 const & — უსაფრთხო დათვალიერება

თუ ფუნქციას მხოლოდ წასაკითხად სჭირდება ობიექტი, ვწერთ `const T &`. კოპირება არ ხდება და ფუნქციას ამ მითითებით ცვლილების უფლებაც არ აქვს.

> **ორი დაზუსტება**
> (1) `const` ზღუდავს ცვლილებას მხოლოდ **ამ მითითებით**; ეს არ ნიშნავს, რომ ობიექტი ყველასთვის უცვლელია: სხვა ადგილიდან, სხვა (არა-`const`) მითითებით, მისი შეცვლა შეიძლება. (2) `const &` ყოველთვის უფრო სწრაფ პროგრამას არ გვაძლევს: პატარა ტიპებისთვის (`int`, `double`) მნიშვნელობით გადაცემა ჩვეულებრივ ისევე კარგია. მთავარი სარგებელი დიდი ობიექტის ზედმეტი კოპირების თავიდან აცილება და კოდის მკითხველისთვის მკაფიო განზრახვაა: „ამ ფუნქციაში ობიექტს მხოლოდ ვკითხულობ“.

C++ perimeter.cpp

```cpp
#include <iostream>
#include <vector>
#include <cmath>

struct Point {
    double x;
    double y;
};

double perimeter(const std::vector<Point> &ring) {
    double total = 0.0;
    for (size_t i = 0; i < ring.size(); ++i) {
        const Point &a = ring[i];
        const Point &b = ring[(i + 1) % ring.size()];
        total += std::hypot(b.x - a.x, b.y - a.y);
    }
    return total;
}

int main() {
    std::vector<Point> parcel = {{0, 0}, {10, 0}, {10, 10}, {0, 10}};
    std::cout << "Perimeter: " << perimeter(parcel) << "\n";
    return 0;
}
```

შედეგი

```
Perimeter: 40
```

ხაზ-ხაზ რას აკეთებს ეს კოდი:

- `struct Point` — წერტილი ორი კოორდინატით (როგორც წინა თავში);
- `std::vector<Point>` — **სია** (vector): ბევრი `Point` ერთად; სიებს თავი 09-ში დაწვრილებით გავეცნობით. აქ ის წარმოადგენს ნაკვეთის კონტურს,

წვეროების თანმიმდევრობას;

- `for (size_t i = 0; i < ring.size(); ++i)` — **ციკლი**: `i` იწყება 0-დან და იზრდება (`++i` ნიშნავს „დაამატე 1“), სანამ წერტილების რაოდენობას არ გაუტოლდება. თითოეულ ნაბიჯზე ვასრულებთ `{ ... }` ბლოკს. `size_t` არის ტიპი, რომელიც რაოდენობებისთვის გამოიყენება;
- `ring[i]` — სიის `i`-ური ელემენტი (ათვლა 0-დან იწყება);
- `a` და `b` — ორი მეზობელი წერტილი, ე.ი. კონტურის ერთი გვერდი;
- `std::hypot(...)` — მანძილი `a`-სა და `b`-ს შორის (თავი 03);
- `total += ...` — გვერდების სიგრძეები ჯამდება.

> **შენიშვნა**
> `(i + 1) % ring.size()` ბოლო წერტილიდან პირველს უკავშირებს ანუ კონტურს ხურავს (`%` არის ნაშთი გაყოფიდან). თუ გადაიცემოდა `std::vector<Point> ring` (`&`-ის გარეშე), ყოველ გამოძახებაზე მთელი წერტილების სია დაკოპირდებოდა.

> **ყურადღება**
> `const`-ის დარღვევა კომპილაციის შეცდომაა. `ring[0].x = 5;` ამ ფუნქციაში კომპილაციას ვერ გაივლის.

05.5 QGIS-ის ხელმოწერების კითხვა

ახლა `api.qgis.org`-ის გვერდებიდან ეს ჩანაწერები ადვილად იკითხება:

ჩანაწერი	როგორ წავიკითხო
`QgsVectorLayer *layer`	ფენის მისამართი; შეიძლება `nullptr` იყოს, შეამოწმე
`const QgsFeature &feature`	ობიექტი აუცილებლად არსებობს, ვკითხულობ და არ ვცვლი, არ ვაკოპირებ
`QgsGeometry &geometry`	ობიექტი არსებობს და ფუნქციას უფლება აქვს, ის შეცვალოს
`const QString &name`	ტექსტი კოპირების გარეშე, მხოლოდ საკითხავად
`QgsFeature feature`	მიიღება ასლი

თუ Python-ს იცნობ, იგივე ფუნქცია ორივე ენაზე:

PYTHON

```python
def count_features(layer):
    return layer.featureCount()
```

C++

```cpp
qint64 countFeatures(const
QgsVectorLayer *layer) {
  return layer->featureCount();
}
```

05.6 სავარჯიშო და შეჯამება

სავარჯიშო

1. დაწერე `void expand(Parcel &p, double extra)`, რომელიც ნაკვეთის ფართობს გაზრდის. შეამოწმე `main`-ში.
2. დაწერე `void printParcel(const Parcel &p)`. შეეცადე მასში ფართობის შეცვლას და ნახე კომპილატორის შეცდომა.
3. დაწერე `double area(const std::vector<Point> &ring)` (შუსტერის ფორმულა: `0.5 * |Σ (xᵢ·yᵢ₊₁ − xᵢ₊₁·yᵢ)|`). კვადრატული ნაკვეთისთვის (0,0), (10,0), (10,10), (0,10) უნდა მიიღო 100.

შეჯამება

- `T &` — მეტსახელი, ობიექტი აუცილებლად არსებობს.
- ფუნქციაში ობიექტს ორიგინალად გადავცემთ `&`-ით, წასაკითხად — `const &`-ით.
- `T *` — მაჩვენებელი, `nullptr`-ს შემოწმებით.
- ახლა `const QgsFeature &feature` სრულად გესმის.

CHAPTER 06

06

კლასები და ობიექტები

Classes and objects

class და struct · public/private · კონსტრუქტორი · const მეთოდები · QGIS-ის სახელდების წესი

QGIS-ში თითქმის ყველაფერი კლასია: `QgsPointXY`, `QgsFeature`, `QgsVectorLayer`. კლასი აერთიანებს **მონაცემს** (წევრ-ცვლადებს) და **ქცევას** (მეთოდებს).

წინა თავში `struct` გავიცანით: ის მხოლოდ მონაცემს აერთიანებს. **კლასი** (class) მას ამატებს ფუნქციებს, რომლებიც ამ მონაცემებზე მუშაობენ. კლასში მოთავსებულ ფუნქციას **მეთოდი** ეწოდება. ობიექტზე ორიენტირებული პროგრამირების (OOP) იდეა ისაა, რომ მონაცემი და მასთან დაკავშირებული ქმედებები ერთად ინახება.

06.1 კლასი და ობიექტი

ნახაზი და სახლი

კლასი არის ნახაზი, ობიექტი კი ამ ნახაზით აშენებული სახლი. ერთი ნახაზით შეიძლება ბევრი სახლი აშენდეს: ერთი კლასი `Parcel`, ბევრი კონკრეტული ნაკვეთი.

ქვემოთ ნაკვეთის მარტივი კლასია. თუ Python-ს იცნობ, შეადარე; თუ არა, უბრალოდ გადახედე C++ მხარეს და ქვემოთ დეტალურ ახსნას წაიკითხე:

PYTHON

```python
class Parcel:
    def __init__(self, owner, w,
h):
        self.owner = owner
        self.w = w
        self.h = h

    def area(self):
        return self.w * self.h
```

C++

```cpp
class Parcel {
public:
  Parcel(std::string owner, double
w, double h);
  double area() const;
private:
  std::string mOwner;
  double mW, mH;
};
```

06.2 public და private

`public` წევრებს ყველა ხედავს, `private` წევრებს მხოლოდ თავად კლასის მეთოდები. გარედან `private`-ზე წვდომა კომპილაციის შეცდომაა. ასე ობიექტი საკუთარ მდგომარეობას იცავს: გარე კოდი მას არასწორ მდგომარეობაში ვერ ჩააგდებს (მაგალითად, ვერ ჩაწერს უარყოფით სიგანეს, თუ ამას მეთოდი ამოწმებს).

> **QGIS-ის წესი**
> წევრ-ცვლადებს აქვთ `m` პრეფიქსი (`mOwner`, `mWidth`), მეთოდები კი `get`-ის გარეშეა: `feature.id()`, არა `feature.getId()`. შეცვლისთვის გამოიყენება `set`: `feature.setGeometry(...)`.

06.3 სრული მაგალითი

C++ parcel.cpp

```cpp
#include <iostream>
#include <string>
#include <utility>

class Parcel {
public:
    Parcel(std::string owner, double width, double height)
        : mOwner(std::move(owner)), mWidth(width), mHeight(height) {}

    double area() const { return mWidth * mHeight; }
    const std::string &owner() const { return mOwner; }

    void resize(double width, double height) {
        mWidth = width;
        mHeight = height;
    }

private:
    std::string mOwner;
    double mWidth;
    double mHeight;
};

int main() {
    Parcel p("Nino", 20.0, 15.0);
    std::cout << p.owner() << ": " << p.area() << " m2\n";

    p.resize(25.0, 20.0);
    std::cout << p.owner() << ": " << p.area() << " m2\n";
    return 0;
}
```

შედეგი

```
Nino: 300 m2
Nino: 500 m2
```

ორი ტერმინი: **წევრი ცვლადი** (member variable) არის ობიექტის შიგნით შენახული მონაცემი (`mOwner`, `mWidth`); **წევრი მეთოდი** (member function) არის ფუნქცია, რომელიც ამ მონაცემებზე მუშაობს (`area()`, `resize()`). ობიექტის შექმნისას კონსტრუქტორი ამზადებს წევრ ცვლადებს, ხოლო მეთოდებს შემდეგ სწორედ ამ ობიექტზე ვუძახებთ.

ხაზ-ხაზ:

კოდი	რას აკეთებს
`class Parcel {`	ვქმნით ახალ ტიპს `Parcel`
`public:`	აქედან დაწყებული ყველაფერი გარედანაც ხელმისაწვდომია
`Parcel(std::string owner, double width, double height)`	**კონსტრუქტორი**: ობიექტის შექმნისას სრულდება და საწყის მნიშვნელობებს უნიშნავს
`: mOwner(std::move(owner)), mWidth(width), ...`	წევრების საწყისი მნიშვნელობები; `std::move` ტექსტს კოპირების გარეშე გადასცემს (ამას თავი 08-ში ავხსნით, ამჟამად ისე გადმოწერე, როგორც წერია)
`double area() const { ... }`	მეთოდი, რომელიც ფართობს გამოთვლის; ბოლოს `const` ნიშნავს, რომ ობიექტს არ ცვლის
`const std::string &owner() const`	მფლობელის სახელს აბრუნებს კოპირების გარეშე
`void resize(...)`	ზომას ცვლის
`private:`	აქ მოთავსებული წევრები მხოლოდ კლასის შიგნიდან ჩანს
`std::string mOwner; double mWidth;...`	ობიექტის მონაცემები
`Parcel p("Nino", 20.0, 15.0);`	`main`-ში ვქმნით ობიექტს და კონსტრუქტორს მნიშვნელობებს ვუგზავნით
`p.area()`	წერტილით ვუძახებთ ობიექტის მეთოდს

06.4 კონსტრუქტორი

კონსტრუქტორი არის სპეციალური მეთოდი, რომელიც ობიექტის შექმნისას სრულდება. მას სახელი კლასის სახელი აქვს და დაბრუნების ტიპი არა. წევრებს ვიწყებთ ე.წ. **ინიციალიზაციის სიით** (`: mOwner(...), mWidth(width)`), ეს მიღებული წესია.

C++

```
Parcel p("Nino", 20.0, 15.0);          // ობიექტი შეიქმნა
```

QGIS-ის მაგალითი:

C++

```
QgsPointXY tbilisi(44.8271, 41.7151);   // x = გრძედი(lon), y = განედი(lat)
double x = tbilisi.x();
```

აქ `tbilisi` არის `QgsPointXY` კლასის ობიექტი, კონსტრუქტორს გადავეცით ორი კოორდინატი, შემდეგ მეთოდით `x()` ავიღეთ პირველი.

> **გაითვალისწინე**
> QGIS-ში წერტილი `(x, y)` = `(გრძედი, განედი)`, ანუ `(lon, lat)`. ბევრ რუკის API-ში პირიქითაა. შეცდომა ძალიან გავრცელებულია.

06.5 const მეთოდი

`double area() const` — ბოლოს `const` ნიშნავს, რომ მეთოდი ობიექტს არ ცვლის. ამიტომ ის `const Parcel &`-ზეც გამოიძახება. გახსოვდეს `const QgsFeature &feature`: მასზე მხოლოდ `const` მეთოდები გამოიძახება.

მეთოდი	`const`?	რატომ
`area()`	დიახ	მხოლოდ კითხულობს
`owner()`	დიახ	მხოლოდ კითხულობს
`resize(...)`	არა	ცვლის ობიექტს

06.6 struct და class

C++-ში `struct` და `class` თითქმის ერთია. ერთადერთი განსხვავება: `struct`-ში წევრები ნაგულისხმევად `public`-ია, `class`-ში `private`. პრაქტიკაში `struct` მარტივ მონაცემებს (`Point`) ვიყენებთ, `class` — ქცევიან ობიექტებს.

06.7 სავარჯიშო და შეჯამება

სავარჯიშო

1. დაამატე `Parcel`-ს მეთოდი `double perimeter() const`.
2. დაამატე `bool isLargerThan(const Parcel &other) const`.
3. შექმენი კლასი `Region`, რომელსაც აქვს სახელი და მოსახლეობა, და მეთოდი `density(double areaKm2) const`.
4. შეეცადე `p.mWidth = 5;`-ს `main`-ში და წაიკითხე კომპილატორის შეტყობინება.

შეჯამება

- კლასი = მონაცემი + მეთოდები; ობიექტი = კლასის კონკრეტული ეგზემპლარი.
- `private` იცავს მდგომარეობას, `public` არის ინტერფეისი.
- QGIS: `mMember`, `name()`, `setName()`.
- `const` მეთოდი ობიექტს არ ცვლის.

CHAPTER 07

07

მემკვიდრეობა და პოლიმორფიზმი

Inheritance and polymorphism

ბაზისური და მემკვიდრე კლასი · virtual / override · dynamic_cast · qobject_cast

ფენების იერარქია QGIS-ის ხერხემალია: `QgsMapLayer` ზოგადი ფენაა, `QgsVectorLayer` და `QgsRasterLayer` მისი კონკრეტული სახეები. ეს თავი ხსნის, როგორ მუშაობს ასეთი სტრუქტურა.

07.1 მემკვიდრეობა

ზოგადი და კონკრეტული

„ტრანსპორტი“ ზოგადი ცნებაა, „ავტობუსი“ და „მატარებელი“ მისი სახეებია. ორივეს აქვს სიჩქარე და მარშრუტი, მაგრამ თითოეულს თავისი თვისებაც აქვს. ფენებიც ასეა: ყველას აქვს სახელი და CRS, ვექტორულს დამატებით ობიექტები, რასტრულს — არხები (band).

```
QgsMapLayer
 ├─ QgsVectorLayer   (ობიექტები, ატრიბუტები)
 └─ QgsRasterLayer   (პიქსელები, არხები)
```

ფენა, რომლისგანაც ვიღებთ, არის **ბაზისური კლასი** (base class, მშობელი). ფენა, რომელიც იღებს, არის **მემკვიდრე კლასი** (derived class, შვილი). C++-ში მემკვიდრეობა იწერება `: public Base`-ით. ეს მნიშვნელოვანია, რომ კოდი არ გავიმეოროთ: საერთო თვისებები (სახელი, CRS) ერთხელ ვწერთ ბაზისურ კლასში.

07.2 virtual და override

თუ მეთოდი `virtual`-ია, გამოძახებისას ირჩევა **რეალური** ობიექტის ვერსია და არა მაჩვენებლის ტიპისა. ამას პოლიმორფიზმი ჰქვია.

C++ layers.cpp

```cpp
#include <iostream>
#include <string>
#include <utility>
#include <vector>

class Layer {
public:
    explicit Layer(std::string name) : mName(std::move(name)) {}
    virtual ~Layer() = default;

    const std::string &name() const { return mName; }
    virtual std::string type() const { return "generic"; }

private:
    std::string mName;
};

class VectorLayer : public Layer {
public:
    VectorLayer(std::string name, int count)
        : Layer(std::move(name)), mCount(count) {}

    std::string type() const override { return "vector"; }
    int featureCount() const { return mCount; }

private:
    int mCount;
};

class RasterLayer : public Layer {
public:
    explicit RasterLayer(std::string name) : Layer(std::move(name)) {}
    std::string type() const override { return "raster"; }
```

```
};

int main() {
    VectorLayer rivers("Rivers", 42);
    RasterLayer dem("DEM");

    std::vector<Layer *> layers = {&rivers, &dem};
    for (Layer *layer : layers) {
        std::cout << layer->name() << " is " << layer->type() << "\n";
    }
    return 0;
}
```

შედეგი

```
Rivers is vector
DEM is raster
```

ხაზ-ხაზ ახსნა:

კოდი	რას აკეთებს
`class Layer { ... }`	ბაზისური კლასი: ყველა ფენისთვის საერთო თვისებები
`explicit Layer(std::string name)`	კონსტრუქტორი (თავი 06); `explicit` კრძალავს შემთხვევით, უნებლიე გარდაქმნას
`virtual ~Layer() = default;`	**დესტრუქტორი**: სპეციალური მეთოდი, რომელიც ობიექტის წაშლისას სრულდება. `virtual` აუცილებელია ბაზისურ კლასში (იხ. ქვემოთ)
`virtual std::string type() const`	მეთოდი, რომელიც შვილებმა შეიძლება თავიანთი ვერსიით ჩაანაცვლონ
`class VectorLayer : public Layer`	`VectorLayer` არის `Layer`-ის მემკვიდრე
`Layer(std::move(name))` კონსტრუქტორის სიაში	მშობლის კონსტრუქტორს გადავცემთ სახელს
`... override`	„ვცვლი მშობლის მეთოდს“; კომპილატორი შეამოწმებს, რომ ასეთი მეთოდი მართლაც არსებობს
`std::vector<Layer *> layers = {&rivers, &dem};`	სია, რომელშიც სხვადასხვა ტიპის ფენებზე მაჩვენებლები დევს, ორივე `Layer *` ტიპისაა
`for (Layer *layer : layers)`	ციკლი სიის ყველა ელემენტზე (თავი 09)
`layer->type()`	გამოძახება მაჩვენებლით (`->`, თავი 04)

ორივე ელემენტი `Layer *`-ია, მაგრამ `type()` თითოეულისთვის საკუთარ პასუხს აბრუნებს. სწორედ ამიტომ ინახავს QGIS ყველა ფენას ერთ სიაში, როგორც `QgsMapLayer *`.

შენიშვნა

`override` არ არის სავალდებულო, მაგრამ კომპილატორს ამოწმებს: თუ მეთოდის სახელი ან პარამეტრი შეეშალა, შეცდომას დაგიბრუნებს.

ვირტუალური დესტრუქტორი. წარმოიდგინე, რომ ობიექტი შექმნილია როგორც `VectorLayer`, მაგრამ მასზე მაჩვენებელი `Layer *` ტიპისაა და ამ მაჩვენებლით ვშლით ობიექტს (`delete layer;`). თუ `~Layer()` `virtual` არ არის, გაეშვება მხოლოდ ბაზისური კლასის დესტრუქტორი და `VectorLayer`-ის ნაწილი სწორად არ გაიწმინდება (გაჟონვა ან განუსაზღვრელი ქცევა). `virtual ~Layer()` უზრუნველყოფს, რომ სწორი, ანუ მემკვიდრის დესტრუქტორიც გაეშვას. წესი: თუ კლასი ბაზისურად გამოიყენება და მასზე მაჩვენებლით ვმუშაობთ, დესტრუქტორი `virtual` უნდა იყოს.

07.3 ტიპის გარდაქმნა: dynamic_cast

სიაში `Layer *` გვაქვს. `featureCount()` კი მხოლოდ `VectorLayer`-ს აქვს. როგორ მივწვდეთ? `dynamic_cast` ამოწმებს რეალურ ტიპს გაშვების დროს.

C++

```
for (Layer *layer : layers) {
    if (auto *vec = dynamic_cast<VectorLayer *>(layer)) {
        std::cout << vec->name() << ": " << vec->featureCount() << "
features\n";
    }
}
```

შედეგი

```
Rivers: 42 features
```

თუ ობიექტი ვექტორული ფენა არ არის, `dynamic_cast` აბრუნებს `nullptr`-ს. ზუსტად ამიტომ ვამოწმებთ `if`-ში: ჯერ ვცდილობთ გარდაქმნას და ვინახავთ `vec`-ში; თუ `vec` არ არის `nullptr`, `if`-ის ბლოკი სრულდება. `auto *vec`

ნიშნავს „ტიპი თვითონ გამოიცანი (`VectorLayer *`)“. **გარდაქმნა** (cast) ნიშნავს ერთი ტიპის მაჩვენებლის მეორე ტიპად ცნობას, როცა ობიექტი ამ ტიპისაა.

07.4 qobject_cast QGIS-ში

QGIS-ის კლასები `QObject`-იდან მოდის (Qt-ის ბაზისური კლასი, თავი 10), ამიტომ იქ `dynamic_cast`-ის უფრო სწრაფი ვერსია გამოიყენება. ის იგივეს აკეთებს, ოღონდ Qt-ის მექანიზმით. თუ Python-ს იცნობ, `isinstance`-ს ჰგავს:

PYTHON

```
layer = iface.activeLayer()
if isinstance(layer,
QgsVectorLayer):
    print(layer.featureCount())
```

C++

```
auto *vec =
qobject_cast<QgsVectorLayer *>(
              iface-
>activeLayer());
if (vec) {
  qDebug() << vec->featureCount();
}
```

ყურადღება

`qobject_cast` აბრუნებს `nullptr`-ს, თუ გარდაქმნა შეუძლებელია, ანუ აქტიური ფენა რასტრულია ან საერთოდ არ არის არჩეული. ყოველთვის შეამოწმე შედეგი `nullptr`-ზე.

	`dynamic_cast`	`qobject_cast`
რომელი ტიპებისთვის	ნებისმიერი პოლიმორფული კლასი (`virtual` მეთოდებით)	მხოლოდ `QObject` -იდან მემკვიდრე კლასები (`Q_OBJECT` მაკროთი)
საიდან არის	C++-ის სტანდარტი	Qt
ხარჯი	შედარებით მეტი	ჩვეულებრივ ნაკლები
რას აბრუნებს, თუ ტიპი არ ემთხვევა	`nullptr` (მაჩვენებლისთვის)	`nullptr`
როგორ ვამოწმებთ	`if (auto *p = dynamic_cast<T *>(x))`	`if (auto *p = qobject_cast<T *>(x))`

იერარქია გადაამოწმე დოკუმენტაციაში

ამ თავში მოცემული `QgsMapLayer → QgsVectorLayer / QgsRasterLayer` იერარქია გამარტივებულია. ყოველთვის გადაამოწმე კონკრეტული კლასის გვერდზე (Inheritance diagram) `api.qgis.org` -ზე: იქ ზუსტად ჩანს, ვისგან მოდის კლასი და რომელ მეთოდებს იღებს მშობლისგან. ზოგ ვერსიაში ფენების დამატებითი ტიპებიცაა (მაგ., mesh, point cloud, vector tile).

07.5 სავარჯიშო და შეჯამება

სავარჯიშო

1. დაამატე `RasterLayer` -ს `bandCount()` და გამოიტანე `dynamic_cast` -ით.
2. შექმენი მესამე სახე `MeshLayer` და `type()` მიანიჭე `"mesh"` .
3. დაწერე ფუნქცია `int totalFeatures(const std::vector<Layer *> &layers)` , რომელიც მხოლოდ ვექტორული ფენების ობიექტებს დაითვლის.

შეჯამება

- მემკვიდრე კლასი იღებს ბაზისური კლასის ყველაფერს და ამატებს საკუთარს.
- `virtual` + `override` = პოლიმორფიზმი.
- ბაზისური ტიპიდან მემკვიდრეზე: `dynamic_cast` (QGIS-ში `qobject_cast`), შედეგი უნდა შემოწმდეს.

CHAPTER 08

08

მეხსიერების მართვა და smart pointers

Memory management and smart pointers

stack და heap · new/delete · memory leak · unique_ptr · shared_ptr · QGIS-ის მფლობელობის წესები

პროგრამა მუშაობისას მეხსიერებას იკავებს: ცვლადებისთვის, ფენებისთვის, გეომეტრიებისთვის. როცა რაიმე აღარ გვჭირდება, ეს მეხსიერება უნდა გავათავისუფლოთ, თორემ პროგრამა თანდათან გაივსება და შეიძლება გაჩერდეს. ზოგ ენაში (მაგალითად, Python-ში) ამას ავტომატურად აკეთებს ე.წ. ნაგვის შემგროვებელი (garbage collector). C++-ში ეს ჩვენი პასუხისმგებლობაა. კარგი ამბავი ისაა, რომ თანამედროვე C++-ში ამას **smart pointer**-ები აკეთებენ.

08.1 სამი ცნება: მფლობელობა, წვდომა, გადაცემა

მეხსიერების უსაფრთხოდ მართვისთვის ეს სამი ცნება უნდა გავარჩიოთ:

ცნება	კითხვა	მაგალითი
მფლობელობა (ownership)	ვის ევალება ობიექტის სიცოცხლის ციკლი, ანუ ვინ წაშლის მას	`unique_ptr`, პროექტი (`QgsProject`), მშობელი `QObject`
წვდომა (access)	რით მივმართავთ ობიექტს, თუ ის ჩვენი საკუთრება არ არის	`T *` ან `T &` მაჩვენებელი/მითითება (თავები 04–05)
გადაცემა (transfer)	როდის იცვლება მფლობელი და რა ხდება ძველ მაჩვენებელთან	`std::move`, `addMapLayer` (პროექტი იღებს ფენას)

ჩვეულებრივი `T *` და `T &` **ობიექტს არ ფლობენ**: ისინი მხოლოდ წვდომას იძლევიან. ფლობა სხვაგან განისაზღვრება და ეს უნდა ვიცოდეთ, სანამ მაჩვენებელს გამოვიყენებთ.

08.2 stack და heap

	stack	heap
რა ინახება	ლოკალური ცვლადები	`new`-ით შექმნილი ობიექტები
ვინ ასუფთავებს	ავტომატურად, scope-ის დასრულებისას	პროგრამისტი (ან smart pointer)
სიჩქარე	ძალიან სწრაფი	ნელი
ზომა	შეზღუდული	დიდი

> **სასტუმრო და საკუთარი სახლი**
> stack სასტუმროა: ოთახი ავტომატურად გათავისუფლდება, როცა წახვალ. heap — საკუთარი ბინაა: შენ ვალდებული ხარ, დროზე დააბრუნო ან გაყიდო, თორემ სამუდამოდ დაკავებული დარჩება (memory leak).

08.3 new და delete

```C++
Polygon *p = new Polygon("Parcel-1");   // heap-ზე შეიქმნა
// ... ვიყენებთ ...
delete p;                               // ხელით უნდა წავშალოთ
```

თუ `delete` დაგავიწყდა, მეხსიერება გაჟონავს (leak). თუ ორჯერ წაშალე ან წაშლის შემდეგ გამოიყენე, პროგრამა დაიმსხვრევა. ხელით `new`/`delete` თანამედროვე კოდში თითქმის აღარ იწერება.

08.4 unique_ptr: ერთადერთი მფლობელი

smart pointer არის ობიექტი, რომელიც მაჩვენებელივით იქცევა, მაგრამ თავად ზრუნავს მეხსიერების გათავისუფლებაზე. `std::unique_ptr` ობიექტს ფლობს და **scope**-ის (ფიგურული ფრჩხილებით შემოსაზღვრული ბლოკის) დასრულებისას ავტომატურად წაშლის.

C++ unique.cpp

```cpp
#include <iostream>
#include <memory>
#include <string>
#include <utility>

struct Polygon {
    std::string name;
    explicit Polygon(std::string n) : name(std::move(n)) {
        std::cout << "created " << name << "\n";
    }
    ~Polygon() { std::cout << "destroyed " << name << "\n"; }
};

int main() {
    {
        auto p = std::make_unique<Polygon>("Parcel-1");
        std::cout << "using " << p->name << "\n";
    }
    std::cout << "after scope\n";
    return 0;
}
```

შედეგი

```
created Parcel-1
using Parcel-1
destroyed Parcel-1
after scope
```

გავარჩიოთ:

- `Polygon` არის struct, რომელსაც აქვს **კონსტრუქტორი** (იბეჭდება „created“) და **დესტრუქტორი** `~Polygon()` (იბეჭდება „destroyed“). დესტრუქტორი ავტომატურად სრულდება, როცა ობიექტი ქრება;

- `std::make_unique<Polygon>("Parcel-1")` ქმნის `Polygon`-ს heap-ზე და აბრუნებს `unique_ptr`-ს. კუთხური ფრჩხილები ტიპს უთითებს: რის მაჩვენებელია;
- `p->name` — წევრთან მისვლა, როგორც ჩვეულებრივ მაჩვენებელთან;
- ბლოკი `{ ... }` იქმნება მხოლოდ იმისთვის, რომ მისი დასასრული „ჩანდეს“: `p` იქ ქრება და ობიექტიც მაშინვე იშლება.

`delete` არსად დაგვიწერია, მაგრამ ობიექტი ზუსტად ბლოკის ბოლოს განადგურდა. ამ პრინციპს RAII ჰქვია: რესურსი ობიექტის სიცოცხლეს ებმება.

08.5 მფლობელობის გადაცემა: std::move

`unique_ptr`-ის კოპირება შეუძლებელია, რადგან მფლობელი ერთია. შეიძლება მხოლოდ **გადაცემა**. ამას `std::move` აკეთებს: „გადაეცი მფლობელობა“ (მანამდე ბევრგან დაწერილი `std::move(name)` ზუსტად ასეა: ტექსტს ახალ ადგილზე გადააქვს, კოპირების გარეშე).

C++

```
auto a = std::make_unique<Polygon>("Parcel-2");
auto b = std::move(a);          // ახლა b ფლობს, a ცარიელია

if (a == nullptr) {
    std::cout << "a is empty\n";
}
```

შედეგი

```
created Parcel-2
a is empty
destroyed Parcel-2
```

ბოლო ხაზი პროგრამის დასრულებისას ჩნდება, როცა `b` იშლება.

QGIS-ში

`std::unique_ptr<QgsAbstractGeometry>` ხშირად გვხვდება: ფუნქცია, რომელიც ახალ გეომეტრიას ქმნის, მის მფლობელობას გვიბრუნებს. ჩანაწერი `std::unique_ptr<...>` ხელმოწერაში ნიშნავს „ეს შენია, შენ უნდა მართო“.

08.6 shared_ptr: ბევრი მფლობელი

როცა ერთ ობიექტს რამდენიმე ადგილი იყენებს, გამოიყენება `std::shared_ptr`. ობიექტი წაიშლება მაშინ, როცა ბოლო მფლობელიც გაქრება (reference counting).

C++ shared.cpp

```cpp
#include <iostream>
#include <memory>
#include <string>
#include <utility>

struct Polygon {
    std::string name;
    explicit Polygon(std::string n) : name(std::move(n)) {
        std::cout << "created " << name << "\n";
    }
    ~Polygon() { std::cout << "destroyed " << name << "\n"; }
};

int main() {
    auto a = std::make_shared<Polygon>("Shared");
    std::cout << a.use_count() << "\n";
    {
        auto b = a;
        std::cout << a.use_count() << "\n";
    }
    std::cout << a.use_count() << "\n";
    return 0;
}
```

შედეგი

```
created Shared
1
2
1
destroyed Shared
```

`use_count()` გვეუბნება, ამჟამად რამდენი მფლობელი ჰყავს ობიექტს. `auto b = a;` ქმნის მეორე მფლობელს, ამიტომ რიცხვი 2 ხდება. შიდა ბლოკის დასრულებისას `b` ქრება და რიცხვი ისევ 1-ია. ობიექტი მხოლოდ მაშინ წაიშლება, როცა ბოლო მფლობელიც წავა (`main`-ის ბოლოს).

08.7 რა ხდება QGIS-ში

QGIS-ს საკუთარი მფლობელობის წესები აქვს და დოკუმენტაცია მათ განმარტავს:

სიტუაცია	წესი
`QgsProject::addMapLayer(layer)`	პროექტი იღებს ფენის მფლობელობას და თვითონ წაშლის
Qt-ის `QObject` ხე	მშობელი შვილებს თავად წაშლის (იხ. თავი 10)
`std::unique_ptr<T>` დაბრუნება	მიმღები მფლობელია
მაჩვენებელი `T *` ხელმოწერაში	ჩვეულებრივ მფლობელობა არ გადადის

> **ყურადღება**
> თუ ფენას `new QgsVectorLayer(...)`-ით ქმნი და პროექტს აძლევ, თავად `delete` აღარ დაწერო. ფენა პროექტის საკუთრებაა და ორმაგი წაშლა პროგრამას დაამსხვრევს. (`addMapLayer`-ს მეორე პარამეტრი აქვს `takeOwnership = true`: ნაგულისხმევად პროექტი ფლობს ფენას.)

> **QGIS-ის ყველა ობიექტი SMART POINTER-ით არ იმართება**
> ეს თავი smart pointer-ებს ასწავლის, მაგრამ არ ნიშნავს, რომ QGIS-ის ყველა ობიექტი `unique_ptr`-ში ან `shared_ptr`-ში უნდა მოვაქციოთ. QGIS-ის ბევრი კლასი `QObject`-ია და მფლობელობას მშობლის ან პროექტის ხის მექანიზმით წყვეტს. `shared_ptr`-ში ასეთი ობიექტის ჩადებამ შეიძლება ორმაგი წაშლა გამოიწვიოს, რადგან ორი სხვადასხვა მექანიზმი ერთსა და იმავე ობიექტს გაანადგურებს. ყოველთვის წაიკითხე დოკუმენტაცია: ვინ ფლობს ობიექტს, როცა მას პარამეტრად გადასცემ ან უკან იღებ.

08.8 სავარჯიშო და შეჯამება

სავარჯიშო

1. გადააკეთე თავი 04-ის `Layer rivers{...}` მაგალითი `std::make_unique<Layer>(...)`-ზე.
2. შექმენი `std::vector<std::unique_ptr<Polygon>>` და ჩაამატე სამი მრავალკუთხედი (`push_back(std::make_unique<...>(...))`). რა თანმიმდევრობით წაიშლებიან?
3. შეეცადე `unique_ptr`-ის კოპირებას (`auto b = a;`) და წაიკითხე შეცდომა.

შეჯამება

- stack ავტომატურია, heap — მფლობელის პასუხისმგებლობა.
- ხელით `new` / `delete` თავიდან აიცილე, გამოიყენე `make_unique` / `make_shared`.
- `unique_ptr` = ერთი მფლობელი, `shared_ptr` = ბევრი.
- QGIS-ში ყოველთვის გაარკვიე, ვინ ფლობს ობიექტს.

CHAPTER 09

09

STL: კონტეინერები და ალგორითმები

The Standard Template Library

vector · map · range-for · lambda · sort · count_if · accumulate

გეო-მონაცემებთან მუშაობისას ბევრი ერთგვაროვანი მონაცემი გვაქვს: მრავალკუთხედის წვეროები, ფენის ობიექტები, რეგიონების მოსახლეობა. მათ შესანახად გვჭირდება **კონტეინერი**: ობიექტი, რომელიც ბევრ მნიშვნელობას ინახავს. მასზე ოპერაციების (დალაგება, ძებნა, ჯამი) შესასრულებლად კი **ალგორითმები** არსებობს.

STL (Standard Template Library) არის C++-ის სტანდარტული ბიბლიოთეკა სწორედ ამ კონტეინერებისა და ალგორითმებისთვის. QGIS ძირითადად Qt-ის კონტეინერებს იყენებს (თავი 10), მაგრამ STL-ის ცოდნა მაინც აუცილებელია: ორივეს ლოგიკა ერთია და ბევრი QGIS ფუნქცია STL ტიპებს აბრუნებს.

09.1 vector — დინამიური მასივი

`std::vector<T>` არის სია: ერთმანეთის გვერდით მოთავსებული ელემენტები, რომლებსაც ნომრებით (ინდექსებით, 0-დან) მივწვდებით და რომელიც საჭიროებისას იზრდება. კუთხურ ფრჩხილებში `<double>` წერია ელემენტის

ტიპი: `std::vector<double>` არის „დიდი რიცხვების სია“, `std::vector<Point>` — „წერტილების სია“. ერთ სიაში ყველა ელემენტი ერთი ტიპისაა.

თუ Python-ს იცნობ, ის `list`-ის ანალოგია:

PYTHON

```python
areas = [12.5, 7.0, 30.5, 4.0]
areas.append(9.0)
print(len(areas))
for a in areas:
    print(a)
```

C++

```cpp
std::vector<double> areas = {12.5,
7.0, 30.5, 4.0};
areas.push_back(9.0);
std::cout << areas.size() << "\n";
for (double a : areas) {
  std::cout << a << "\n";
}
```

ოპერაცია	Python	C++
დამატება	`append(x)`	`push_back(x)`
ზომა	`len(a)`	`a.size()`
ელემენტი	`a[0]`	`a[0]`
ბოლო	`a[-1]`	`a.back()`
გასუფთავება	`a.clear()`	`a.clear()`

> **წესი**
> დიდი ელემენტების ციკლში გამოიყენე `const auto &`: `for (const auto &feature : features)`. ასე ელემენტი არ დაკოპირდება (თავი 05).

09.2 map — გასაღები → მნიშვნელობა

`std::map<K, V>` ინახავს წყვილებს „გასაღები → მნიშვნელობა“, მაგალითად „ქალაქი → მოსახლეობა“. გასაღებით მნიშვნელობას სწრაფად ვპოულობთ, გასაღებებს კი კონტეინერი ავტომატურად ალაგებს ზრდადობით (Python-ის `dict`-ის მსგავსია). კუთხურ ფრჩხილებში ორი ტიპია: გასაღებისა და მნიშვნელობის.

C++ map.cpp

```cpp
#include <iostream>
#include <map>
#include <string>

int main() {
    std::map<std::string, int> population = {
        {"Tbilisi", 1200000},
        {"Batumi", 170000},
        {"Kutaisi", 147000},
    };

    population["Rustavi"] = 125000;

    for (const auto &[city, people] : population) {
        std::cout << city << ": " << people << "\n";
    }
    return 0;
}
```

შედეგი

```
Batumi: 170000
Kutaisi: 147000
Rustavi: 125000
Tbilisi: 1200000
```

`std::map` გასაღებებს **დალაგებულად** ინახავს. თუ დალაგება არ გჭირდება და მხოლოდ სწრაფი ძებნა გაინტერესებს, არსებობს `std::unordered_map`: მისი ელემენტების თანმიმდევრობა წინასწარ განსაზღვრული არ არის.

ახსნა: `population["Rustavi"] = 125000;` ამატებს ახალ წყვილს. ციკლში `for (const auto &[city, people] : population)` ყოველი წყვილიდან ორივე ნაწილს ცალ-ცალკე ვიღებთ (`city` და `people`); ეს ჩანაწერი C++17-ის სიახლეა და მას **structured binding** ჰქვია. `const auto &` კი ნიშნავს „ტიპი თვითონ გამოიცანი, არ დააკოპირო და არ შეცვალო“ (თავი 05).

შენიშვნა

მოსახლეობის რიცხვები აქ მიახლოებითი და საილუსტრაციოა, მიზანი კონტეინერის ჩვენებაა.

ყურადღება

`population["Poti"]` — თუ გასაღები არ არსებობს, `map` ჩუმად შექმნის ახალ ჩანაწერს ნულოვანი მნიშვნელობით. არსებობის შესამოწმებლად გამოიყენე `population.find("Poti") != population.end()` ან `population.count("Poti")`.

09.3 ალგორითმები

`<algorithm>` და `<numeric>` გვაძლევს მზა ოპერაციებს: დალაგება, დათვლა, ჯამი.

C++ algo.cpp

```cpp
#include <algorithm>
#include <iostream>
#include <numeric>
#include <vector>

int main() {
    std::vector<double> areas = {12.5, 7.0, 30.5, 4.0};

    std::sort(areas.begin(), areas.end(),
              [](double a, double b) { return a > b; });

    for (double a : areas) std::cout << a << " ";
    std::cout << "\n";

    double total = std::accumulate(areas.begin(), areas.end(), 0.0);
    std::cout << "Total: " << total << "\n";
    std::cout << "Average: " << total / areas.size() << "\n";

    long big = std::count_if(areas.begin(), areas.end(),
                             [](double a) { return a > 10.0; });
    std::cout << "Parcels > 10 ha: " << big << "\n";
    return 0;
}
```

შედეგი

```
30.5 12.5 7 4
Total: 54
Average: 13.5
Parcels > 10 ha: 2
```

ახსნა: ალგორითმები სიას **დიაპაზონის** სახით იღებენ: `areas.begin()` არის დასაწყისი, `areas.end()` — „ბოლოს მერე“. `std::sort` ალაგებს; მესამე არგუმენტი ამბობს, როგორ შევადაროთ ორი ელემენტი (აქ „უფრო დიდი წინ

იყოს“, ამიტომ დალაგება კლებადია). `std::accumulate` აჯამებს (დაწყება `0.0`-დან). `std::count_if` ითვლის, რამდენი ელემენტი აკმაყოფილებს პირობას.

09.4 lambda — უსახელო ფუნქცია

`[](double a, double b) { return a > b; }` არის lambda: პატარა ფუნქცია, რომელიც ადგილზევე იწერება და სახელი არ სჭირდება. `[]` არის დასაწყისი, შემდეგ პარამეტრები, შემდეგ ფუნქციის სხეული. (Python-ში ეს `lambda a, b: a > b`-ს ჰგავს.) `[]` კვადრატულ ფრჩხილებში შეიძლება გარე ცვლადების „დაჭერა“:

```C++
double limit = 10.0;
auto isBig = [limit](double a) { return a > limit; };
```

QGIS-ში lambda-ებს ხშირად გამოიყენებენ signal-ებთან დასაკავშირებლად (თავი 10).

09.5 ძებნა

```C++
auto it = std::find_if(areas.begin(), areas.end(),
                       [](double a) { return a > 20.0; });
if (it != areas.end()) {
    std::cout << "First > 20: " << *it << "\n";
}
```

`find_if` აბრუნებს **iterator-ს**: ელემენტზე მიმთითებელს (მაჩვენებლის მსგავს ობიექტს). თუ ელემენტი ვერ მოიძებნა, შედეგი უდრის `areas.end()`-ს, ამიტომ `*it`-ს გამოყენებამდე ეს ყოველთვის უნდა შევამოწმოთ (როგორც `nullptr` მაჩვენებელზე). იგივე ეხება `std::max_element`-ს: ცარიელ სიაზე ის `end()`-ს აბრუნებს. `*it` იღებს მნიშვნელობას.

ცარიელი კონტეინერი

`std::accumulate` ცარიელ სიაზე საწყის მნიშვნელობას (`0.0`) დააბრუნებს, ხოლო საშუალოს გამოთვლისას `total / areas.size()` ნულზე გაყოფა იქნება. ამიტომ გაყოფამდე ყოველთვის შეამოწმე `if (!areas.empty())`.

QGSFEATUREITERATOR STL-ის კონტეინერი არ არის

QGIS-ში `QgsFeatureIterator` ამავე იდეაზეა აგებული (ობიექტებს სათითაოდ გვაწვდის), მაგრამ ის STL-ის კონტეინერი არ არის: ობიექტები მეხსიერებაში წინასწარ არ ინახება, ისინი მონაცემთა პროვაიდერიდან (ფაილიდან ან ბაზიდან) წამოდის და მხოლოდ ერთხელ გავივლით. ამიტომ მას `begin()` / `end()` და STL ალგორითმები ჩვეულებრივ არ აქვს; იყენებენ `nextFeature(...)` ციკლს.

09.6 სავარჯიშო და შეჯამება

სავარჯიშო

1. დაწერე `std::vector<Point>` ხუთი წერტილით და `std::sort`-ით დაალაგე x-ის მიხედვით.
2. შექმენი `std::map<std::string, double>`: მდინარე → სიგრძე კმ-ში (`{"Mtkvari", 1364}`, `{"Rioni", 327}`, `{"Alazani", 407}`). იპოვე ყველაზე გრძელი (`std::max_element` მიმართე დოკუმენტაციას).
3. `count_if`-ით დათვალე მდინარეები, რომლებიც 400 კმ-ზე გრძელია.

შეჯამება

- `vector` = list, `map` = dict, ორივე ტიპიზებულია.
- ალგორითმები იღებენ დიაპაზონს `begin()` – `end()` და lambda-ს.
- iterator — მაჩვენებლის მსგავსი ობიექტი კონტეინერში გადასაადგილებლად.

CHAPTER 10

10

Qt: QString, QVariant, signals და slots

Qt essentials for QGIS

QString · QList · QVariant · QgsAttributes · QObject-ის ხე · signal/slot · lambda connect

Qt არის C++-ის ფართო ბიბლიოთეკა („ფრეიმვორკი“): ფანჯრები, ღილაკები, ტექსტი, კონტეინერები, ფაილები და მრავალი სხვა. QGIS Qt-ზეა აგებული, ამიტომ API-ში `std::string`-ს თითქმის ვერ ნახავ: ყველგან `QString`, `QList`, `QVariant`. ამ თავში იმ მინიმუმს ვისწავლით, რომელიც QGIS-ის კოდის წასაკითხად საჭიროა.

Qt-ის კლასები `Q`-ზე იწყება და დიდი ნაწილი თავის header ფაილში ცხოვრობს (`#include <QString>`). QGIS-ის საკუთარი კლასები კი `Qgs`-ზე იწყება.

10.1 QString

`QString` არის ტექსტის ტიპი, ისევე როგორც `std::string`, მაგრამ Qt-ისთვის გაწყობილი. ის Unicode ტექსტს ინახავს, ამიტომ ქართული ასოებიც პრობლემის გარეშე მუშაობს და ბევრი მოხერხებული ოპერაცია აქვს.

```C++
QString name = QStringLiteral("მტკვარი");
QString info = QString("%1: %2 კმ").arg(name).arg(1364);


qDebug() << info.length();
```

- `QStringLiteral("...")` — ტექსტი, რომელიც კომპილაციისას მზადდება (ეფექტურობისთვის, ამჟამად ჩვეულებრივ ბრჭყალებად წაიკითხე);
- `%1` და `%2` — ადგილები, რომლებშიც `.arg(...)` მნიშვნელობებს ჩასვამს თანმიმდევრობით; შედეგი: `მტკვარი: 1364 კმ`;
- `qDebug() << ...` — Qt-ის გზა, ტექსტი დიაგნოსტიკისთვის კონსოლში გამოვიტანოთ (როგორც `std::cout`).

ოპერაცია	Python	Qt
სიგრძე	`len(s)`	`s.length()`
ნაწილი	`s[2:5]`	`s.mid(2, 3)`
ჩანაცვლება	`s.replace(a, b)`	`s.replace(a, b)`
ფორმატირება	`f"{a}: {b}"`	`QString("%1: %2").arg(a).arg(b)`
რიცხვად	`int(s)`	`s.toInt()`, `s.toDouble()`
std::string-ში		`s.toStdString()`

> **ქართული და STD::STRING**
> `toStdString()` UTF-8-ს აბრუნებს. კონსოლში ქართული ტექსტი მხოლოდ მაშინ გამოჩნდება სწორად, თუ ტერმინალი UTF-8-ზეა. `qDebug()`-ით დაბეჭდვისას ტექსტი ბრჭყალებშია (`"..."`), `qDebug().noquote()` კი ბრჭყალებს გაუქმებს.

10.2 QList

`QList<T>` არის Qt-ის სია, `std::vector`-ის ახლობელი (თავი 09): ზომა საჭიროებისას იზრდება, ელემენტებს ინდექსით მივწვდებით. `<T>` ისევ ელემენტის ტიპია.

QList და std::vector ერთი და იგივე არ არის

მათი API და მეხსიერების ორგანიზაცია განსხვავდება და Qt-ის ვერსიებს შორისაც იცვლება: Qt6-ში `QList` უკვე თანმიმდევრულად (ერთმანეთის გვერდით) ინახავს ელემენტებს, Qt5-ში კი ეს ყოველთვის ასე არ იყო. ამიტომ ერთს მეორეთი ავტომატურად ნუ ჩაანაცვლებ; კონკრეტული მეთოდები და გარანტიები კონკრეტულ დოკუმენტაციაში გადაამოწმე.

C++

```
QList<QString> cities = {"Tbilisi", "Batumi"};
cities.append("Kutaisi");
for (const QString &c : cities) {
    qDebug() << c;
}
```

გამოძახება `append`, `size()`, `[]`, `contains()`. QGIS-ში ფენების სია არის `QList<QgsMapLayer *>`, მაგალითად `QgsProject::mapLayersByName(...)` სწორედ ამას აბრუნებს.

10.3 QVariant — უნივერსალური ყუთი

`QVariant` ერთ მნიშვნელობას ინახავს ნებისმიერი ტიპისას: რიცხვი, ტექსტი, თარიღი, ან „არაფერი“ (NULL).

C++ variant.cpp

```cpp
#include <QString>
#include <QVariant>
#include <iostream>

int main() {
    QVariant population = 1200000;
    QVariant city = QString("Tbilisi");
    QVariant lat = 41.7151;
    QVariant unknown;                    // NULL

    std::cout << population.toInt() << "\n";
    std::cout << city.toString().toStdString() << "\n";
    std::cout << lat.toDouble() << "\n";
    std::cout << std::boolalpha << unknown.isNull() << "\n";
    return 0;
}
```

შედეგი

```
1200000
Tbilisi
41.7151
true
```

დაიმახსოვრე

`QVariant` არ „იცის“ წინასწარ, რა ტიპია შიგნით, ამიტომ კითხვისას თავად ვირჩევთ `toInt()`, `toString()`, `toDouble()`. თუ გარდაქმნა შეუძლებელია, შედეგი ნულოვანი ან ცარიელია.

წარუმატებელი გარდაქმნა და ნამდვილი ნული

`QVariant("abc").toInt()` დაგიბრუნებს `0`-ს, ისევე როგორც `QVariant(0).toInt()`. როგორ გაარჩიო? `toInt()`-ს აქვს არასავალდებულო პარამეტრი `bool *ok`:

```
C++
bool ok = false;
int value = v.toInt(&ok);
if (!ok) {
    // გარდაქმნა ვერ მოხერხდა; ეს ნამდვილი ნული არ არის
}
```

ატრიბუტებთან მუშაობისას ეს განსაკუთრებით მნიშვნელოვანია: ველი შეიძლება იყოს NULL, ცარიელი ტექსტი ან მართლა `0`, და სამივე სხვადასხვა რამეს ნიშნავს.

10.4 QgsAttributes — ატრიბუტების ცხრილის ერთი სტრიქონი

ახლა გასაგებია `QgsAttributes` გვერდი, რომლითაც კურსი დაიწყო. ატრიბუტების ცხრილში ობიექტის ერთი სტრიქონი სხვადასხვა ტიპის მნიშვნელობებისგან შედგება (ტექსტი, რიცხვი, ათწილადი, NULL). ამიტომ `QgsAttributes` არის `QVariant`-ების მიმდევრობა: ფენის ცხრილის ერთი ჩანაწერი. თითოეული ელემენტი ერთი ველის მნიშვნელობაა.

C++ attrs.cpp

```cpp
#include <QList>
#include <QString>
#include <QVariant>
#include <iostream>

int main() {
    // ველები: name, length_km, comment
    QList<QVariant> attrs = {
        QVariant(QString("Mtkvari")),
        QVariant(1364),
        QVariant(),
    };

    for (const QVariant &v : attrs) {
        std::cout << (v.isNull() ? "NULL" : v.toString().toStdString()) <<
"\n";
    }
    return 0;
}
```

შედეგი

```
Mtkvari
1364
NULL
```

ახსნა: სიაში სამი ელემენტი ძევს: ტექსტი, რიცხვი და ცარიელი `QVariant()` (უცნობი მნიშვნელობა, NULL). ციკლში `for (const QVariant &v : attrs)` თითოეულს გადავხედავთ; `v.isNull()` ამოწმებს, ცარიელია თუ არა; `v.toString()` ნებისმიერ მნიშვნელობას ტექსტად გადააქცევს; `? :` არის „თუ... მაშინ... თორემ...“: `პირობა ? თუ_ჭეშმარიტია : თუ_მცდარია`.

თუ Python-ს იცნობ, შედარება:

PYTHON

```python
feature = layer.getFeature(1)
print(feature.attributes())
# ['Mtkvari', 1364, NULL]
```

C++

```cpp
QgsFeature feature = layer->getFeature(1);
QgsAttributes attrs = feature.attributes();
QVariant name = attrs.at(0);
```

10.5 QObject-ის ხე

`QObject` არის Qt-ის ბაზისური კლასი: მისგან მოდის ყველა ფანჯარა, ღილაკი და QGIS-ის ბევრი კლასიც (`QgsMapLayer`, `QgsProject`). მისი მნიშვნელოვანი თვისებაა, რომ `QObject`-ები მშობელ-შვილის ხეში ცხოვრობენ. როცა მშობელი იშლება, თავის შვილებსაც წაშლის. ამიტომ Qt-ში ხშირად `new` გვხვდება `delete`-ის გარეშე (კონტრასტია თავი 08-ის წესებთან: აქ მფლობელი მშობელი ობიექტია). ეს წესი მხოლოდ მაშინ მუშაობს, თუ მშობელ-შვილის კავშირი მართლა დაყენებულია (მაგალითად, კონსტრუქტორში მშობელი გადაეცა). `new QPushButton("OK")` მშობლის გარეშე ავტომატურად არავის შვილი არ არის და მისი წაშლა შენს საქმეა.

C++

```cpp
auto *dialog = new QDialog();
auto *button = new QPushButton("OK", dialog);   // dialog არის მშობელი
// delete dialog;  ->  button-იც ავტომატურად წაიშლება
```

10.6 signals და slots

გრაფიკულ პროგრამაში ბევრი რამ მოვლენების საპასუხოდ ხდება: მომხმარებელმა დააჭირა ღილაკს, პროექტს დაემატა ფენა. Qt ამას ხსნის **signal/slot** მექანიზმით: signal არის „რაღაც მოხდა“, slot — „ამაზე რეაგირება“. `connect` მათ აკავშირებს.

```C++
connect(QgsProject::instance(), &QgsProject::layersAdded,
        this, &MyTool::onLayersAdded);
```

ან lambda-თი, დამატებითი მეთოდის გარეშე:

```C++
connect(button, &QPushButton::clicked, this, [this]() {
    qDebug() << "clicked";
});
```

`connect(გამომგზავნი, &კლასი::signal, მიმღები, &კლასი::slot)`: ოთხი არგუმენტია. მეორე მაგალითში ბოლო არგუმენტი lambda-ა (თავი 09), ანუ ადგილზე დაწერილი პატარა ფუნქცია, რომელიც ღილაკზე დაჭერისას შესრულდება. `this` არის მიმდინარე ობიექტი, რომელშიც ვიმყოფებით.

> **QT-ის კოდის აწყობა**
> Qt-ზე დაწერილ კოდს აწყობისას Qt-ის ბიბლიოთეკები სჭირდება (თავი 12-ში CMake-ით დავაკავშირებთ). ამ თავის მაგალითები QGIS-ის გარეშეც იკომპილება, თუ Qt დაგიყენებია.

> **საფოსტო ყუთი**
> signal არის ზარის ხმა კარზე: ის არავის ეუბნება, ვინ გაიგონებს. slot არის ის, ვინც ზარზე რეაგირებს. `connect` ამათ ერთმანეთს აკავშირებს და გამომგზავნს მიმღები არ აინტერესებს.

PyQGIS-ში იგივეა: `layer.selectionChanged.connect(handler)`.

10.7 სავარჯიშო და შეჯამება

სავარჯიშო

1. შექმენი `QList<QVariant>` ფენის ჩანაწერისთვის: სახელი (ტექსტი), მოსახლეობა (`int`), ფართობი (`double`). დაბეჭდე თითოეული ელემენტი.
2. შექმენი `QMap<QString, QVariant>` იგივე მონაცემით და გამოიტანე ველი სახელით.
3. წაიკითხე `QgsFeature::attribute` მეთოდი api.qgis.org-ზე და ჩამოწერე მისი ორი ვარიანტი.

შეჯამება

- `QString` = Unicode ტექსტი, `QList<T>` = მასივი, `QVariant` = უნივერსალური მნიშვნელობა.
- `QgsAttributes` = `QVariant` -ების მიმდევრობა.
- `QObject` ხე თავად ასუფთავებს შვილებს.
- signal/slot აკავშირებს „მოხდა“ და „რეაგირება“ ერთმანეთს.

CHAPTER 11

11

api.qgis.org-ის კითხვა

Reading the QGIS C++ API documentation

კლასის გვერდის სტრუქტურა · ხელმოწერების გაშიფვრა · სასარგებლო კლასები · სტრატეგია

დოკუმენტაცია (api.qgis.org) თავიდან გაუგებარი ჩანს, რადგან ის პროგრამისტისთვისაა დაწერილი და ხშირად ტერმინებს არ განმარტავს. ახლა უკვე გაქვს ყველა საჭირო ინსტრუმენტი: ტიპები, მაჩვენებლები, მითითებები, კლასები, `QVariant`. ამ თავში სისტემატურად ვისწავლით დოკუმენტაციის „კითხვას“.

ეს გვერდები ავტომატურად გენერირდება კოდის header (`.h`) ფაილებიდან ინსტრუმენტით **Doxygen**, ამიტომ დოკუმენტაცია ზუსტად ის არის, რაც კოდშია. რამდენიმე სასარგებლო ტერმინი: **enum** — დასახელებული მნიშვნელობების ნაკრები (მაგალითად, ფენის ტიპები: ვექტორული, რასტრული); **typedef** — არსებული ტიპის მეორე სახელი; **static** მეთოდი — მეთოდი, რომელსაც ობიექტის გარეშე, პირდაპირ კლასიდან ვუძახებთ.

11.1 კლასის გვერდის სქემა

`api.qgis.org/api/<ვერსია>/class<სახელი>.html` გვერდს ერთი და იგივე სტრუქტურა აქვს:

ნაწილი	რას ეძებ იქ
Detailed Description	რისთვისაა კლასი, მაგალითები; ყოველთვის ჯერ ეს წაიკითხე
Inheritance diagram	ვისგან მოდის (იხ. თავი 07); მემკვიდრეობით მიღებული მეთოდები მშობლის გვერდზეა
Public Types	enum-ები და typedef-ები; აქ ჩანს, რას ნიშნავს კლასის სახელი
Public Member Functions	მეთოდები; ძირითადი სამუშაო
Static Public Member Functions	გამოიძახება კლასზე, ობიექტის გარეშე: `QgsProject::instance()`
Signals / Slots	რა მოვლენებს აგზავნის
Member Function Documentation	თითოეული მეთოდის დეტალები, `Since QGIS 3.x` ნიშნები

> **წესი**
> პირველად გვერდის ბოლოს ნუ იწყებ. წაიკითხე **Detailed Description**, შემდეგ მეთოდების სია, შემდეგ კონკრეტული მეთოდი.

11.2 ხელმოწერის გაშიფვრა

ავიღოთ რამდენიმე რეალური ჩანაწერი და წავიკითხოთ ისინი ისე, როგორც კომპილატორი კითხულობს.

ჩანაწერი	კლასი	როგორ იკითხება
`QgsGeometry geometry() const`	`QgsFeature`	ობიექტის გეომეტრია, ასლად; მეთოდი ობიექტს არ ცვლის
`void setGeometry(const QgsGeometry &geometry)`	`QgsFeature`	გეომეტრიას ვუთითებ; კოპირება არ ხდება, ორიგინალს არ შეცვლის
`QVariant attribute(const QString &name) const`	`QgsFeature`	ველის მნიშვნელობა სახელით; `QVariant`-ად
`QgsFeatureId id() const`	`QgsFeature`	ობიექტის უნიკალური ნომერი
`bool isValid() const`	`QgsFeature`	ობიექტი ვალიდურია თუ არა
`QgsFeatureIterator getFeatures(const QgsFeatureRequest &request = QgsFeatureRequest()) const`	`QgsVectorLayer`	ობიექტების iterator; პარამეტრი არასავალდებულოა (default)
`double area() const`	`QgsGeometry`	გეომეტრიის გეგმური ფართობი (კოორდინატების ერთეულის კვადრატი)
`static QgsGeometry fromPointXY(const QgsPointXY &point)`	`QgsGeometry`	`static`: გამოიძახება `QgsGeometry::fromPointXY(...)`-ით
`QList<QgsMapLayer *> mapLayersByName(const QString &name) const`	`QgsProject`	ფენების სია სახელით; ელემენტები მაჩვენებლებია, შეამოწმე ცარიელია თუ არა

11.3 მაკროები, რომლებიც შეგხვდება

ჩანაწერი	აზრი
`SIP_SKIP`, `SIP_OUT`, `SIP_FACTORY`	მითითებები Python-ის დამაკავშირებელი გენერატორისთვის (SIP); C++-ში შეგიძლია უგულებელყო
`Q_INVOKABLE`	მეთოდი Qt-ის მეტა-სისტემიდან გამოიძახება
`Q_DECL_DEPRECATED`	მოძველებულია, ნუ გამოიყენებ
`CORE_EXPORT`	კლასი ბიბლიოთეკიდან გამოდის; ტექნიკური დეტალია
`Since QGIS 3.xx`	მეთოდი ამ ვერსიიდან არსებობს

11.4 ცოცხალი მაგალითი: ამოცანიდან მეთოდამდე

ამოცანა: ფენაში თითოეული ობიექტის ფართობი გამოვთვალოთ.

1. იპოვე ფენის კლასი: `QgsVectorLayer`.
2. იქ ნახე `getFeatures()` → აბრუნებს `QgsFeatureIterator`-ს.
3. იტერატორიდან მიიღებ `QgsFeature`-ს.
4. `QgsFeature::geometry()` → `QgsGeometry`.
5. `QgsGeometry::area()` → `double`.

```C++
QgsFeatureIterator it = layer->getFeatures();
QgsFeature feature;
while (it.nextFeature(feature)) {
    double area = feature.geometry().area();
    qDebug() << feature.id() << area;
}
```

ხაზ-ხაზ: `getFeatures()` იძლევა იტერატორს (ობიექტებს სათითაოდ გვაწვდის); `QgsFeature feature;` ცარიელი ობიექტია, რომელშიც ყოველ ნაბიჯზე ახალი მოხვდება; `while (it.nextFeature(feature))` იმეორებს

ციკლს, სანამ ობიექტები არსებობს; ციკლის შიგნით `feature.geometry().area()` ჯერ გეომეტრიას იღებს, შემდეგ მის ფართობს; `qDebug()` კი შედეგს ბეჭდავს.

თუ Python-ს იცნობ, შედარება:

PYTHON

```python
for f in layer.getFeatures():
    print(f.id(),
f.geometry().area())
```

C++

```cpp
QgsFeature f;
QgsFeatureIterator it = layer-
>getFeatures();
while (it.nextFeature(f)) {
  qDebug() << f.id() <<
f.geometry().area();
}
```

ფართობი და CRS

`QgsGeometry::area()` დოკუმენტაციის მიხედვით გეგმური (planar) ფართობია, გამოთვლილი კოორდინატების ერთეულებში. გეოგრაფიულ CRS-ში (გრადუსები) ის კვადრატული გრადუსია და კვადრატულ მეტრებად არ ითვლება. გეოდეზიური (ელიფსოიდური) გამოთვლისთვის დოკუმენტაცია `QgsDistanceArea`-ს გვირჩევს (იხ. თავი 12). ასეთი ნიუანსები ხშირად დოკუმენტაციის აღწერაშია, ამიტომ ხელმოწერასთან ერთად აღწერაც წაიკითხე.

11.5 ვერსიები

ამ თავის ხელმოწერები გადამოწმებულია `api.qgis.org`-ის **4.3.0-Master** დოკუმენტაციასთან. თუ შენი QGIS სხვა ვერსიისაა (მაგ. 3.x), გვერდის ზედა ნაწილში ვერსიის არჩევით გადაამოწმე იგივე კლასი: ზოგ მეთოდს განსხვავებული overload შეიძლება ჰქონდეს. მაგალითად, `QgsGeometry::buffer` ახალ დოკუმენტაციაში ორი სახითაა: `buffer(double distance, int segments, QgsFeedback *feedback = nullptr) const` და გაფართოებული ვერსია სტილის პარამეტრებით.

11.6 ბილიკი დოკუმენტაციიდან კოდამდე

ვთქვათ, გვინდა ფენის ობიექტების რაოდენობა. ეტაპები:

1. გახსენი `QgsVectorLayer`-ის გვერდი და მოძებნე „count“. იპოვე `long long featureCount() const`;
2. გვერდის ზედა ნაწილში წერია header: `#include <qgsvectorlayer.h>`;
3. ეს კლასი `qgis_core` ბიბლიოთეკაშია (იხ. თავი 01, არქიტექტურის ცხრილი), ამიტომ CMake-ში `qgis_core` უნდა მივაბათ;
4. გამოიყენე: `long long n = layer->featureCount();` (მაჩვენებელზე `->`, ობიექტზე `.`).

ანუ ყოველ ახალ მეთოდზე სამი რამ გვჭირდება: **ხელმოწერა**, `#include` და **ბიბლიოთეკა**, რომელშიც ის დევს.

11.7 ძირითადი კლასები ერთ ცხრილში

კლასი	რას აკეთებს	PyQGIS-ის ანალოგი
`QgsProject`	პროექტი, ფენების მფლობელი	`QgsProject.instance()`
`QgsVectorLayer`	ვექტორული ფენა	იგივე
`QgsRasterLayer`	რასტრული ფენა	იგივე
`QgsFeature`	ობიექტი: ID, გეომეტრია, ატრიბუტები	იგივე
`QgsGeometry`	გეომეტრია და ოპერაციები	იგივე
`QgsPointXY`	2D წერტილი	იგივე
`QgsCoordinateReferenceSystem`	CRS	იგივე
`QgsCoordinateTransform`	გადაყვანა CRS-ებს შორის	იგივე
`QgsFeatureRequest`	ფილტრი ობიექტების არჩევისას	იგივე

11.8 შეცდომის კითხვა

კომპილატორის შეცდომა პირველად საშიშად გამოიყურება, მაგრამ ხშირად ერთი ხაზი გვეუბნება მთავარს:

შეცდომა	ჩვეულებრივი მიზეზი
`'QgsFeature' was not declared`	ჩასართავი `#include <qgsfeature.h>` გაგრჩა
`undefined reference to ...`	ბიბლიოთეკა არ არის მიბმული (`target_link_libraries`)
`passing 'const X' as 'this' discards qualifiers`	`const` ობიექტზე არა-`const` მეთოდი გამოიძახე
`cannot convert 'QString' to 'std::string'`	გამოიყენე `.toStdString()`

11.9 სავარჯიშო და შეჯამება

სავარჯიშო

1. გახსენი `QgsAttributes` გვერდი. იპოვე მშობელი კლასი და ჩამოწერე სამი მეთოდი.
2. იპოვე `QgsGeometry::buffer` ხელმოწერა. რა პარამეტრები აქვს? რას აბრუნებს? რომელი პარამეტრია `const`?
3. იპოვე, რა მეთოდი უნდა გამოიყენო ფენაში არჩეული ობიექტების დასათვლელად (`QgsVectorLayer`-ის გვერდზე ეძებე „selected“).
4. გადააკეთე წინა მაგალითი ისე, რომ მხოლოდ ისეთი ობიექტები დაითვალოს, რომელთა ფართობი 1000-ზე მეტია.

შეჯამება

- ჯერ **Detailed Description**, მერე მეთოდების სია.
- ხელმოწერა იკითხება მარცხნიდან: დაბრუნების ტიპი, სახელი, პარამეტრები (`*` / `&` / `const`), მეთოდის `const`.
- PyQGIS-ის ცოდნა = დოკუმენტაციის ცოდნა: სახელები ერთია.

CHAPTER 12

12

პროექტი: საკუთარი ინსტრუმენტი QGIS-ში

Capstone — your own tool inside QGIS

დამოუკიდებელი C++ აპლიკაცია · CRS და ფართობი · CMake · C++ პლაგინის სტრუქტურა · QGIS-ის აწყობა · ლიცენზია

საბოლოო მიზანი: საკუთარი C++ კოდი QGIS-თან ერთად გაუშვა. გზა სამ საფეხურად ვაგებთ: ჯერ ვმუშაობთ QGIS-ის ბიბლიოთეკასთან დამოუკიდებელ აპლიკაციაში, შემდეგ ვაკეთებთ პლაგინის სტრუქტურას, ბოლოს (არასავალდებულოდ) ვაწყობთ QGIS-ს წყაროდან.

ამ თავში შემოდის ახალი ტერმინები: **პლაგინი** არის დამატება, რომელსაც QGIS ჩატვირთვისას აკავშირებს თავის მენიუსა და ფუნქციებთან; C++ პლაგინი არის **დინამიური ბიბლიოთეკა** (`.dll` Windows-ზე, `.so` Linux-ზე) — ცალკე ფაილი, რომელიც უშუალოდ კი არ იხსნება, არამედ სხვა პროგრამა (QGIS) ტვირთავს. **CMake** კი არის „აწყობის დირიჟორი“: ტექსტური ფაილი (`CMakeLists.txt`), რომელიც ეუბნება, რომელი ფაილები და ბიბლიოთეკები დააკომპილოს და როგორ დააკავშიროს.

გაიგე, რა არის და რა არ არის შემოწმებული
ამ თავის კოდი შედგენილია QGIS-ის API-ის დოკუმენტაციისა და QGIS-ის რეალური წყაროს (`src/plugins/topology`) მიხედვით, მაგრამ ავტორს ის ჯერ არ აუწყვია და არ გაუშვია ყველა გარემოში. გამოიყენე ის როგორც გზამკვლევი, პირველად გაუშვი ერთ კონკრეტულ სისტემაზე და სისტემის ბილიკები და ვერსიები მოარგე შენს ინსტალაციას. API-ის ხელმოწერები შემოწმებულია `api.qgis.org` -ის 4.3.0-Master დოკუმენტაციასთან; QGIS 3.x-ში ზოგი overload შეიძლება განსხვავდებოდეს.

12.1 გეგმა

საფეხური	შედეგი	სირთულე
1	კონსოლური პროგრამა, რომელიც `qgis_core` -ს იყენებს და პოლიგონების ფართობს ითვლის	დაბალი
2	C++ პლაგინის სტრუქტურა: რა ფაილებისგან შედგება და როგორ ერთვება QGIS-ში	საშუალო
3	QGIS-ის აწყობა წყარო კოდიდან (გაფართოებული, არასავალდებულო)	მაღალი

რჩევა
ბევრი პირდაპირ მესამე საფეხურზე ხტება და აწყობის დეტალებში იჭედება. პირველი ორი უკვე გასაგებ შედეგს იძლევა და გასწავლის ბიბლიოთეკების ერთად მუშაობას.

12.2 CRS და ფართობი: ჯერ ეს უნდა გავიგოთ

ფართობის გამოთვლა მხოლოდ ფორმულა არ არის. **CRS** (კოორდინატთა სისტემა) განსაზღვრავს, რას ნიშნავს კოორდინატის რიცხვი: გრადუსს თუ მეტრს. `QgsGeometry::area()` ითვლის **გეგმურ (დეკარტულ) ფართობს**

კოორდინატების ერთეულებში. თუ მონაცემები გეოგრაფიულ CRS-შია (მაგალითად, WGS 84, გრადუსებში), შედეგი **კვადრატული გრადუსია** და კვადრატულ მეტრებად ვერ ჩაითვლება.

გზა	როდის	შედეგის ერთეული
`geometry().area()`	მონაცემები უკვე საპროექციო CRS-შია (მეტრებში, მაგ. UTM)	CRS-ის ერთეულის კვადრატი
`QgsDistanceArea::measureArea(...)`	ზუსტი ელიფსოიდური გამოთვლა, ნებისმიერი CRS-ისთვის	დამოკიდებულია `areaUnits()`-ზე (ელიფსოიდის გამოყენებისას კვადრატული მეტრი)
გადაპროექტება `QgsCoordinateTransform`-ით	გვინდა გეომეტრია სხვა CRS-ში	ახალი CRS-ის ერთეული

> **რუკა და მასშტაბი**
> რუკაზე „5 სანტიმეტრი“ არაფერს ნიშნავს, სანამ მასშტაბი არ ვიცით. კოორდინატის რიცხვიც ასეა: მნიშვნელობა მხოლოდ CRS-ით მიიღება.

12.3 საფეხური 1: პოლიგონების ფართობი qgis_core-ზე

ამოცანა: რაიონების (პოლიგონების) ფენის ჩატვირთვა, ობიექტების დათვლა და ფართობების ჯამის გამოთვლა ელიფსოიდურად.

C++ main.cpp

```cpp
#include <iostream>

#include <qgsapplication.h>
#include <qgsdistancearea.h>
#include <qgsfeature.h>
#include <qgsgeometry.h>
#include <qgsproject.h>
#include <qgsvectorlayer.h>

int main(int argc, char *argv[]) {
    QgsApplication app(argc, argv, false);     // false = GUI არ გვჭირდება
    QgsApplication::setPrefixPath("/usr", true); // QGIS-ის ინსტალაციის
საქაღალდე
    QgsApplication::initQgis();

    QgsVectorLayer layer("districts.geojson", "districts", "ogr");
    if (!layer.isValid()) {
        std::cerr << "Layer failed to load\n";
        return 1;
    }

    // ელიფსოიდური ფართობი: წყარო CRS + ელიფსოიდი
    QgsDistanceArea da;
    da.setSourceCrs(layer.crs(), QgsProject::instance()->transformContext());
    da.setEllipsoid("WGS84");

    std::cout << "Features: " << layer.featureCount() << "\n";

    double planar = 0.0;      // კოორდინატების ერთეულის კვადრატი
    double ellipsoidal = 0.0; // კვადრატული მეტრი
    QgsFeature feature;
    QgsFeatureIterator it = layer.getFeatures();
    while (it.nextFeature(feature)) {
```

```
        planar += feature.geometry().area();
        ellipsoidal += da.measureArea(feature.geometry());
    }
    std::cout << "Planar sum: " << planar << " (CRS units^2)\n";
    std::cout << "Ellipsoidal sum: " << ellipsoidal << " m2\n";

    QgsApplication::exitQgis();
    return 0;
}
```

გამოსავალი ასე შეიძლება გამოიყურებოდეს (რიცხვები საილუსტრაციოა და შენს მონაცემებზეა დამოკიდებული):

შედეგი

```
Features: 12
Planar sum: 0.0382741 (CRS units^2)
Ellipsoidal sum: 453812345 m2
```

თუ ფენა გეოგრაფიულ CRS-შია, პირველი რიცხვი პატარაა (კვადრატული გრადუსი), მეორე კი კვადრატული მეტრებია. სწორედ ამიტომ ორივეს ერთად ვბეჭდავთ.

ხაზ-ხაზ:

კოდი	რას აკეთებს
`#include <qgs...h>`	ვრთავთ QGIS-ის კლასებს (header ფაილები, თავი 03)
`int main(int argc, char *argv[])`	`main`-ის ვერსია, რომელიც ბრძანების ხაზის არგუმენტებს იღებს (Qt მათ ითხოვს)
`QgsApplication app(argc, argv, false);`	QGIS-ის „გარემოს“ ობიექტი; `false` ნიშნავს, რომ გრაფიკული ინტერფეისი არ გვჭირდება (headless, ქვეთავი 1.3)
`QgsApplication::setPrefixPath("/usr", true);`	ვეუბნებით, სად დგას QGIS (პლაგინები, რესურსები); Linux-ზე სისტემური ინსტალაციისას ჩვეულებრივ `/usr`
`QgsApplication::initQgis();`	QGIS-ის ინიციალიზაცია (static მეთოდია: კლასიდან გამოიძახება); ამზადებს მონაცემთა პროვაიდერებს, CRS ბაზას და სხვას
`QgsVectorLayer layer("districts.geojson", "districts", "ogr");`	ვქმნით ფენას: ფაილი, სახელი, მონაცემთა პროვაიდერი (`ogr` კითხულობს ბევრ ფორმატს)
`if (!layer.isValid())`	თუ ფენა ვერ ჩაიტვირთა, ვბეჭდავთ შეცდომას `std::cerr`-ით და ვასრულებთ პროგრამას კოდით `1`
`QgsDistanceArea da; da.setSourceCrs(...); da.setEllipsoid("WGS84");`	ზუსტი გაზომვის ობიექტი: ვუთითებთ ფენის CRS-ს და ელიფსოიდს
`da.measureArea(geometry)`	გეომეტრიის ფართობი ელიფსოიდზე
`while (it.nextFeature(feature))`	ციკლი ობიექტებზე (თავი 11)
`QgsApplication::exitQgis();`	გამართულად ვასრულებთ მუშაობას და ვათავისუფლებთ რესურსებს

> **შენიშვნა**
> `setPrefixPath` ბილიკი სისტემაზეა დამოკიდებული: Linux-ის პაკეტიდან დაყენებისას ჩვეულებრივ `/usr`, ხელით აწყობისას — `--prefix`-ის მნიშვნელობა, Windows-ზე (OSGeo4W) — QGIS-ის საქაღალდე. თუ ბილიკი არასწორია, ფენა `isValid() == false` დაგიბრუნებს.

12.4 CMake: ერთი კონკრეტული გარემო

ქვემოთ მაგალითი **Ubuntu/Debian-ისთვისაა**, სადაც QGIS 3.x და Qt5 სისტემური პაკეტებით დგას. QGIS 4-სთვის გამოიყენება Qt6, ამიტომ `Qt5` უნდა შეიცვალოს `Qt6`-ით. სხვა სისტემებზე ბილიკები და პაკეტების სახელები განსხვავდება.

დამოკიდებულებების დაყენება:

```shell
sudo apt install build-essential cmake qgis libqgis-dev qtbase5-dev
```

`CMakeLists.txt`:

CMAKE `CMakeLists.txt`

```
cmake_minimum_required(VERSION 3.16)
project(district_area LANGUAGES CXX)
set(CMAKE_CXX_STANDARD 17)
set(CMAKE_CXX_STANDARD_REQUIRED ON)

# QGIS 3.x = Qt5. QGIS 4 = Qt6: შეცვალე Qt5 -> Qt6.
find_package(Qt5 REQUIRED COMPONENTS Core Gui Xml Widgets)

find_path(QGIS_INCLUDE_DIR qgsapplication.h PATH_SUFFIXES qgis)
find_library(QGIS_CORE_LIBRARY qgis_core)

if(NOT QGIS_INCLUDE_DIR OR NOT QGIS_CORE_LIBRARY)
  message(FATAL_ERROR "QGIS headers or qgis_core not found")
endif()

add_executable(district_area main.cpp)
target_include_directories(district_area PRIVATE ${QGIS_INCLUDE_DIR})
target_link_libraries(district_area PRIVATE
    ${QGIS_CORE_LIBRARY} Qt5::Core Qt5::Gui Qt5::Xml Qt5::Widgets)
```

ფაილის ნაწილები: `project(...)` ამლევს პროექტს სახელს; `find_package`, `find_path`, `find_library` ეძებენ საჭირო ბიბლიოთეკებს სისტემაში (პირობა `if(NOT ...)` გვაჩერებს, თუ ვერაფერი იპოვა); `add_executable` ამბობს, რომ ვაწყობთ პროგრამას `main.cpp`-დან; `target_link_libraries` აკავშირებს მას QGIS-სა და Qt-თან.

SHELL

```
cmake -S . -B build
cmake --build build
cd /path/to/data      # საქაღალდე, სადაც districts.geojson დევს
/path/to/build/district_area
```

პირველი ბრძანება ამზადებს აწყობას `build` საქაღალდეში, მეორე აკომპილებს და აკავშირებს (compile + link), მესამე უშვებს შედეგს. პროგრამა `districts.geojson`-ს ეძებს იმ საქაღალდეში, საიდანაც გაუშვი (ან მიუთითე სრული გზა კოდში).

ტიპური შეცდომები და მათი გამოსწორება

შეცდომა	მიზეზი	რა გააკეთო
`qgsapplication.h: No such file or directory`	კომპილატორმა QGIS-ის header-ები ვერ იპოვა	დააყენე `libqgis-dev` და შეამოწმე `QGIS_INCLUDE_DIR`
`undefined reference to QgsApplication::...`	`qgis_core` არ არის მიბმული	შეამოწმე `target_link_libraries`
`libqgis_core.so.3.xx: cannot open shared object file`	გაშვებისას ბიბლიოთეკა ვერ მოიძებნა	დაამატე ბილიკი `LD_LIBRARY_PATH`-ში ან `RPATH`-ში
`Layer failed to load`	არასწორი ფაილის გზა ან `setPrefixPath`	შეამოწმე ფაილის გზა და `setPrefixPath`-ის მნიშვნელობა
Qt5/Qt6 შეუსაბამობა	პროგრამა სხვა Qt ვერსიით აიწყო, ვიდრე QGIS	გამოიყენე ის Qt, რომელზეც QGIS არის აწყობილი

12.5 საფეხური 2: C++ პლაგინი QGIS-ის წყაროში

C++ პლაგინი არის დინამიური ბიბლიოთეკა, რომელსაც QGIS გაშვებისას ტვირთავს. QGIS-ის რეპოზიტორიაში უკვე არსებობს რეალური C++ პლაგინები (`src/plugins/topology`, `geometry_checker`, `offline_editing`). საუკეთესო სასწავლო გზაა მათი სტრუქტურის გამეორება. ქვემოთ მინიმალური პლაგინია, აგებული `topology`-ს მიხედვით.

პლაგინის სავალდებულო ნაწილები (QGIS-ის `qgisplugin.h`-იდან):

1. კლასი, რომელიც მემკვიდრეობს `QgisPlugin`-იდან და ახორციელებს `initGui()` და `unload()` მეთოდებს;

2. `extern "C"` ფუნქციები: `classFactory`, `name`, `description`, `category`, `type`, `version`, `icon`, `unload`. QGIS მათ პლაგინის ბიბლიოთეკიდან ეძახის სახელით, ამიტომ ისინი C++-ის სახელების გადაკეთების (name mangling) გარეშე უნდა გამოჩნდეს; ამას მაკრო `QGISEXTERN` აკეთებს;
3. `CMakeLists.txt`, რომელიც `MODULE` ტიპის ბიბლიოთეკას ქმნის.

C++ mytool.h

```cpp
#ifndef MYTOOL_H
#define MYTOOL_H

#include <QObject>
#include "../qgisplugin.h"

class QAction;
class QgisInterface;

class MyTool : public QObject, public QgisPlugin {
    Q_OBJECT
public:
    explicit MyTool(QgisInterface *iface);

public slots:
    void initGui() override;     // QGIS გვიძახებს ჩატვირთვისას
    void unload() override;      // და გამორთვისას
    void run();                  // მენიუს პუნქტის დაჭერისას

private:
    QgisInterface *mIface = nullptr;
    QAction *mAction = nullptr;
};

#endif
```

C++ mytool.cpp

```cpp
#include "qgisinterface.h"
#include "qgsapplication.h"
#include "qgsmaplayer.h"
#include "qgsvectorlayer.h"

#include <QAction>
#include <QMessageBox>
#include <QString>

#include "mytool.h"
#include "moc_mytool.cpp"

static const QString sName = QObject::tr("Feature Counter");
static const QString sDescription = QObject::tr("Counts features of the
active vector layer");
static const QString sCategory = QObject::tr("Vector");
static const QString sPluginVersion = QObject::tr("Version 0.1");
static const QgisPlugin::PluginType sPluginType = QgisPlugin::UI;
static const QString sPluginIcon = QString();

MyTool::MyTool(QgisInterface *iface)
    : QgisPlugin(sName, sDescription, sCategory, sPluginVersion,
sPluginType),
      mIface(iface) {}

void MyTool::initGui() {
    delete mAction;
    mAction = new QAction(sName, this);
    connect(mAction, &QAction::triggered, this, &MyTool::run);
    mIface->addPluginToMenu(tr("&My Tools"), mAction);
}

void MyTool::unload() {
```

```
    mIface->removePluginMenu(tr("&My Tools"), mAction);
    delete mAction;
    mAction = nullptr;
}

void MyTool::run() {
    auto *layer = qobject_cast<QgsVectorLayer *>(mIface->activeLayer());
    if (!layer) {
        QMessageBox::warning(nullptr, tr("Error"), tr("Select a vector
layer."));
        return;
    }
    QMessageBox::information(nullptr, tr("Result"),
        tr("Features: %1").arg(layer->featureCount()));
}

// ---- სავალდებულო ექსპორტირებული ფუნქციები ----
QGISEXTERN QgisPlugin *classFactory(QgisInterface *iface) { return new
MyTool(iface); }
QGISEXTERN const QString *name() { return &sName; }
QGISEXTERN const QString *description() { return &sDescription; }
QGISEXTERN int type() { return sPluginType; }
QGISEXTERN const QString *category() { return &sCategory; }
QGISEXTERN const QString *version() { return &sPluginVersion; }
QGISEXTERN const QString *icon() { return &sPluginIcon; }
QGISEXTERN void unload(QgisPlugin *plugin) { delete plugin; }
```

CMAKE CMakeLists.txt

```
set(mytool_SRCS mytool.cpp)

add_library(plugin_mytool MODULE ${mytool_SRCS})

include_directories(
  ${CMAKE_SOURCE_DIR}/src/plugins
  ${CMAKE_BINARY_DIR}/src/ui
  ${CMAKE_CURRENT_BINARY_DIR}
)

target_link_libraries(plugin_mytool qgis_core qgis_gui)

install(TARGETS plugin_mytool
  RUNTIME DESTINATION ${QGIS_PLUGIN_DIR}
  LIBRARY DESTINATION ${QGIS_PLUGIN_DIR})
```

ფაილები `mytool/` საქაღალდეში უნდა დაიდოს QGIS-ის წყაროში `src/plugins/mytool/`, ხოლო `src/plugins/CMakeLists.txt`-ში უნდა დაემატოს ხაზი `add_subdirectory(mytool)` (იქ ახლა წერია `add_subdirectory(topology)`, `add_subdirectory(geometry_checker)` და სხვა). შემდეგ QGIS-ის აწყობა (საფეხური 3) შენს პლაგინსაც ააგებს.

ხაზ-ხაზ: `class MyTool : public QObject, public QgisPlugin` — მემკვიდრეობა `QObject`-იდან (signal/slot-ისთვის) და `QgisPlugin`-იდან (პლაგინის ინტერფეისი). `Q_OBJECT` არის Qt-ის მაკრო, რომელიც signal/slot მექანიზმს რთავს. `initGui()` QGIS-ს ჩატვირთვისას ეძახის: აქ ვქმნით მენიუს პუნქტს (`QAction`) და ვაკავშირებთ `run()` მეთოდთან. `unload()` გამორთვისას ასუფთავებს. `run()` კი ინსტრუმენტის ლოგიკაა: იღებს აქტიურ ფენას, ამოწმებს, რომ ვექტორულია, და შეტყობინებით აჩვენებს ობიექტების რაოდენობას. `iface` (`QgisInterface`) QGIS-ის ფანჯრასთან ურთიერთობის შესასვლელია. `tr("...")` აღნიშნავს ტექსტს, რომლის თარგმნაც მომავალში იქნება შესაძლებელი.

> **QGIS-ის წყაროს ჩვევები**
> `#include "moc_mytool.cpp"` ბოლოს და `../qgisplugin.h` იმ კონვენციებს მიჰყვება, რომელსაც `src/plugins/topology/topol.cpp` იყენებს. თუ შენს QGIS ვერსიაში აწყობა სხვა ფორმას მოითხოვს, პირდაპირ `topology` პლაგინს შეადარე: ის ყოველთვის სწორია ამ ვერსიისთვის.

12.6 საფეხური 3: QGIS-ის აწყობა წყაროდან (გაფართოებული)

ეს საფეხური არ არის აუცილებელი წიგნის ძირითადი შედეგისთვის. QGIS-ის აწყობა სერიოზული ამოცანაა: მოითხოვს დამოკიდებულებებს (Qt, GDAL, GEOS, PROJ და სხვა), მნიშვნელოვან დროსა და ადგილს დისკზე. დაიწყე მხოლოდ მაშინ, როცა პირველი ორი საფეხური უკვე მუშაობს.

```shell
git clone https://github.com/qgis/QGIS.git
cd QGIS
mkdir build && cd build
cmake -G Ninja -DCMAKE_BUILD_TYPE=Debug ..
ninja
```

ეტაპი	რა ხდება
`cmake`	ამოწმებს დამოკიდებულებებს და ქმნის build ფაილებს; დამოკიდებულებების სრული სია QGIS-ის `INSTALL.md`-შია
`ninja`	აკომპილებს; პირველად დიდ დროს მოითხოვს
ტესტი	გაუშვი აწყობილი `qgis`; პლაგინი ჩატვირთე „Plugin Manager“-იდან

მცირე ცვლილების შეტანის შემდეგ `ninja` მხოლოდ შეცვლილ ფაილებს აკომპილებს.

ლიცენზია

QGIS ვრცელდება GNU GPL ლიცენზიით. ამ ლიცენზიის მოთხოვნები (მათ შორის წყარო კოდის გადაცემის ვალდებულება) დამოკიდებულია იმაზე, **როგორ და ვისთვის** ავრცელებ პროგრამას, ხოლო კონკრეტული შემთხვევა შეიძლება გარემოებებზე იყოს დამოკიდებული. ეს ტექსტი იურიდიული რჩევა არ არის. თუ გეგმავ საკუთარი კოდის გავრცელებას QGIS-თან ერთად ან მასში ჩაშენებულს, წაიკითხე ლიცენზიის პირობები და საჭიროებისას მიმართე იურისტს. გაითვალისწინე ისიც, რომ კომპილირებული ბინარული კოდის დაცვა აბსოლუტური არ არის: მისი უკუინჟინერია რთულია, მაგრამ შესაძლებელია.

12.7 სავარჯიშო და შეჯამება

საბოლოო დავალება

1. გააკეთე საფეხური 1 შენი მონაცემით (მაგ. რაიონების პოლიგონები). **მოსალოდნელი შედეგი:** პროგრამა ბეჭდავს ობიექტების რაოდენობას და ორ ფართობს; ელიფსოიდური ფართობი კვადრატულ მეტრებშია და შესამჩნევად განსხვავდება „Planar sum“-ისგან, თუ ფენა გეოგრაფიულ CRS-შია.
2. შეამოწმე შედეგი QGIS-ის გრაფიკულ ინტერფეისში (ატრიბუტთა ცხრილის გამოსახულება `$area`) და შეადარე.
3. დაამატე `QgsFeatureRequest` ფილტრი და დათვალე მხოლოდ ობიექტები გარკვეული ატრიბუტით. **შემოწმება:** რაოდენობა უნდა დაემთხვეს QGIS-ში ამავე პირობით არჩეულ ობიექტებს.
4. წაიკითხე `src/plugins/topology/topol.cpp` და იპოვე იქ ყველა `QGISEXTERN` ფუნქცია. **შემოწმება:** ჩამოწერე რვავე სახელი.
5. მოამზადე 5-ხაზიანი დოკუმენტი: რა ისწავლე, რა გაუგებარია, რა გინდა შემდეგ ეტაპზე.

კურსის შეჯამება

- PyQGIS და C++ API ერთი ბირთვის ორი ინტერფეისია; სახელები უმეტესად ერთია, ზუსტი ხელმოწერები კი ყოველთვის უნდა გადაამოწმო.
- `*` — შეიძლება `nullptr`, `&` — აუცილებლად არსებობს, `const &` — უსაფრთხო წაკითხვა.
- smart pointer-ები და QObject-ის ხე მეხსიერებას თავად მართავენ, მაგრამ QGIS-ს საკუთარი მფლობელობის წესები აქვს.
- `QgsGeometry::area()` გეგმურია; გეოგრაფიული ფართობისთვის `QgsDistanceArea`.
- `api.qgis.org` იკითხება: აღწერა, ტიპები, ხელმოწერა.
- გზა: დამოუკიდებელი აპლიკაცია → პლაგინის სტრუქტურა → QGIS-ის აწყობა.